

Assignment 2: PaaS with GitHub deployment (Full version)

Course: Cloud Computing & Data Engineering

By: Hamed Ahmadinia

Introduction

This assignment focuses on deploying a containerized web service to the cloud using GitHub-based continuous deployment and a Platform-as-a-Service (PaaS) provider. The goal of the assignment is to demonstrate practical skills in using GitHub for version control, building a simple API with FastAPI, containerizing the application with Docker, and deploying it automatically to the cloud using Render.

The assignment workflow follows the steps introduced in the course lectures. A FastAPI application was created using the provided docker-fastapi template repository. The application was deployed to Render directly from GitHub, enabling automatic redeployment whenever changes were committed and pushed to the repository. The repository was cloned locally and opened in Visual Studio Code, where the API response was modified and tested both locally and in the cloud.

Docker was used to test the application locally before deployment, ensuring that the service behaves consistently across environments. In addition to the required tasks, optional extensions were implemented, including a compute endpoint that processes client requests and a machine learning-based wine prediction API deployed as a separate service.

All source code and configuration files used in this assignment are available in the following GitHub repositories:

- FastAPI deployment repository:
- <https://github.com/Hamed-Ahmadinia/paas-fastapi-Cloud-Computing>
- Wine prediction demo repository:
- <https://github.com/Hamed-Ahmadinia/wine-prediction-demo>

This report documents each step of the development and deployment process, supported by screenshots, and concludes with reflections on the challenges encountered and lessons learned during the assignment.

Part A – FastAPI Repository Setup: In Part A, a new GitHub repository was created using the provided docker-fastapi template. The template repository was copied using GitHub's "Use this template" feature, and a new public repository named paas-fastapi-Cloud-Computing was created under my GitHub account. After creating the repository, I verified that all required files from the template were present, including the FastAPI application structure, Dockerfile, Docker Compose configuration, and dependency files. This repository served as the base for all further development and deployment steps in the assignment.

Issue encountered: No major issues were encountered during this step.

Resolution: The repository was created successfully on the first attempt and was ready for local cloning and further modifications.

Assignment 2: PaaS with GitHub deployment (Full version)

Course: Cloud Computing & Data Engineering

By: Hamed Ahmadiania

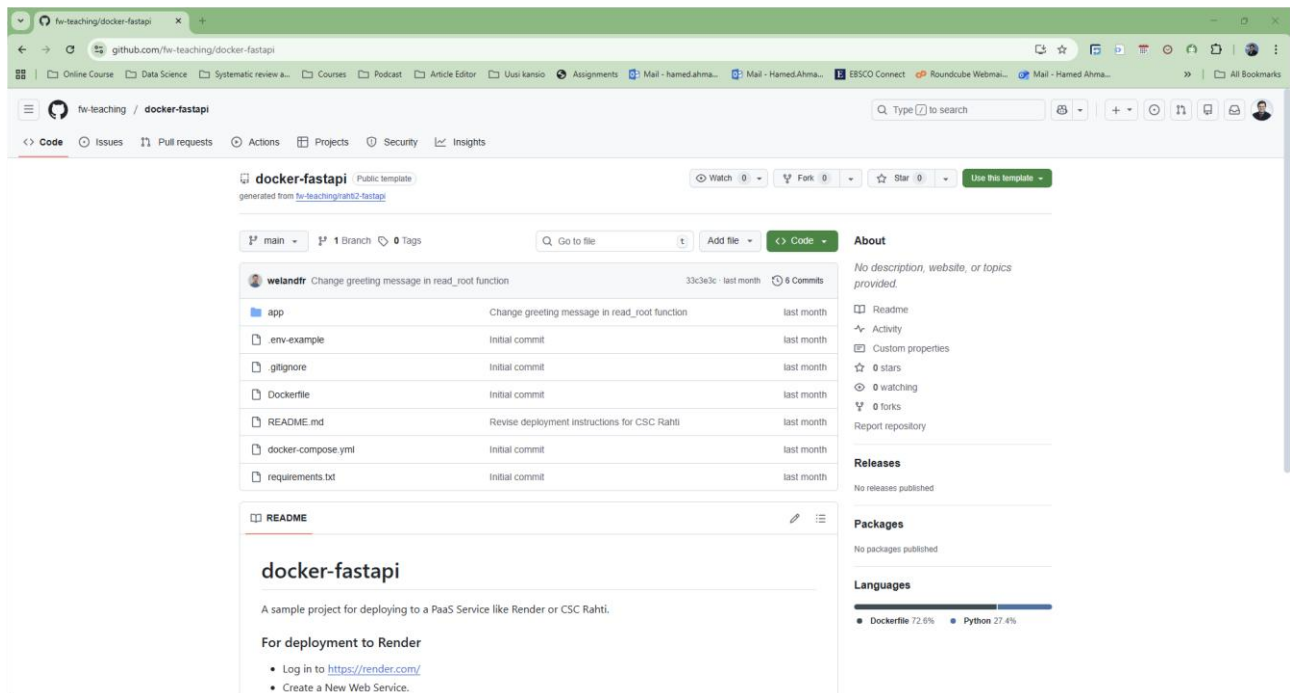


Figure 1: GitHub template repository (fw-teaching/docker-fastapi) used to create the FastAPI project

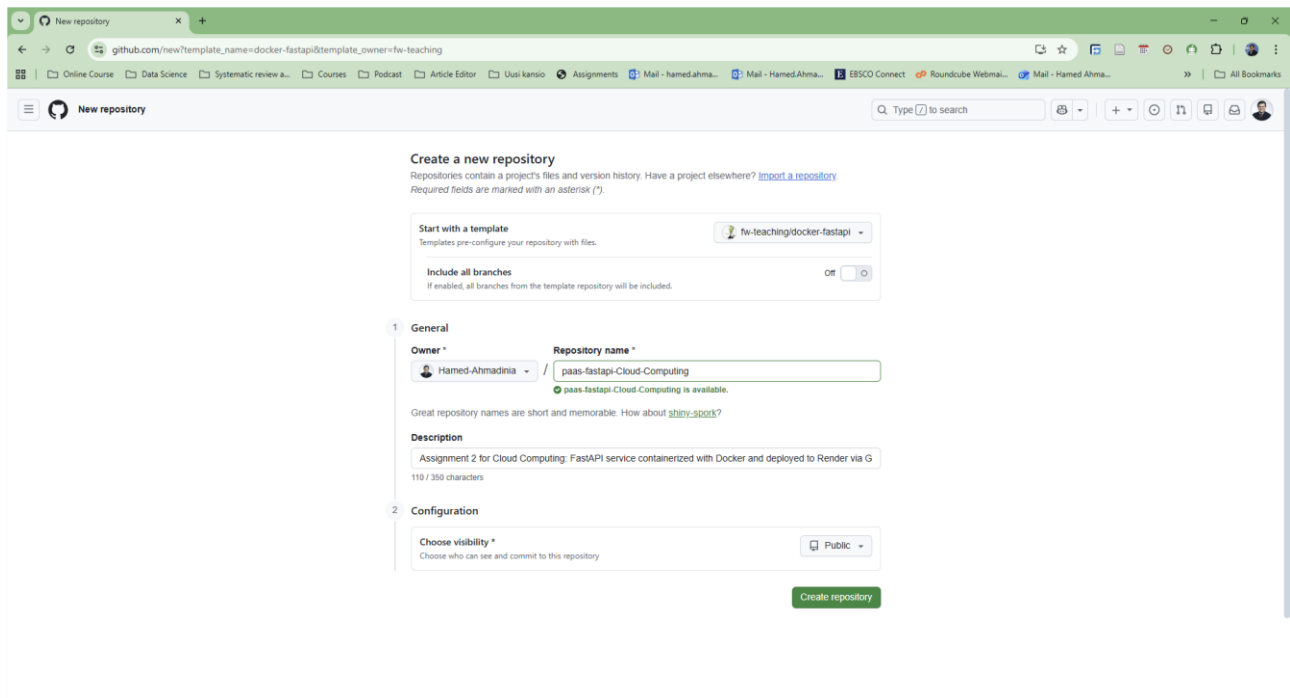


Figure 2: Creating my FastAPI project repository from the provided GitHub template.

Assignment 2: PaaS with GitHub deployment (Full version)

Course: Cloud Computing & Data Engineering

By: Hamed Ahmadiania

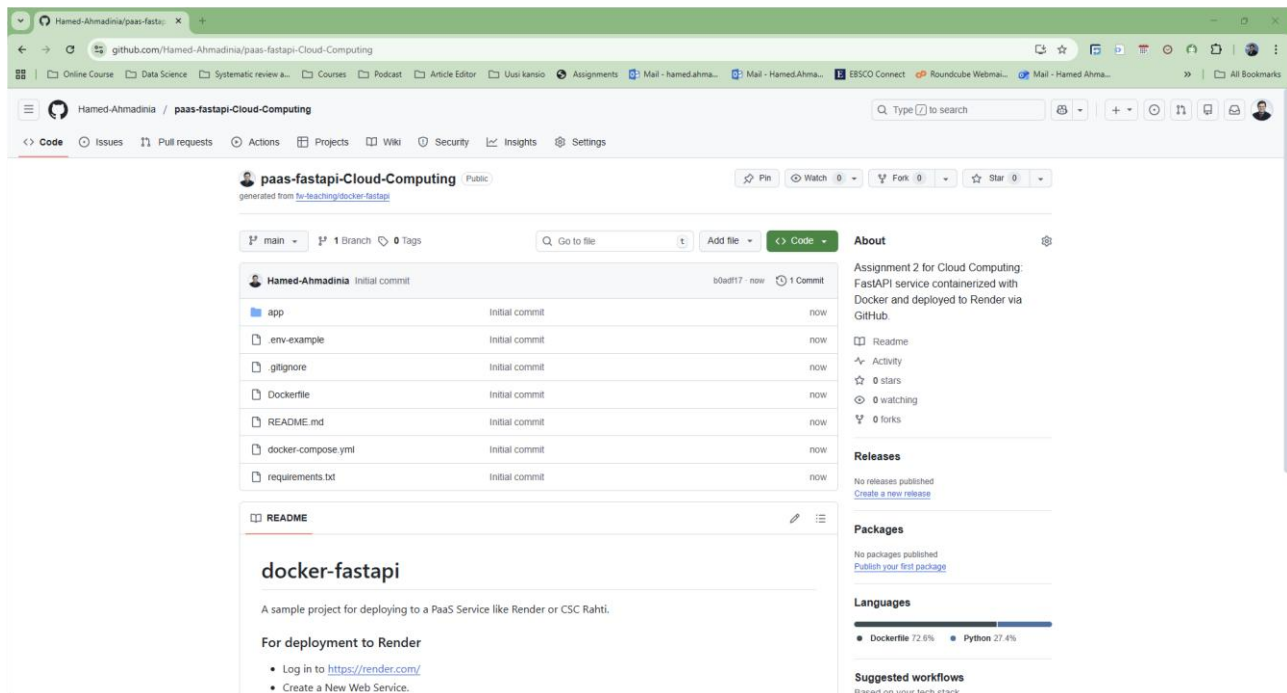


Figure 3: My FastAPI project repository after being created from the provided template.

Part B – Deployment to Render (PaaS): In Part B, I deployed the FastAPI application to the cloud using Render as a Platform-as-a-Service (PaaS). I logged in to Render using my GitHub account and created a new Web Service. During setup, I connected Render to GitHub and granted access only to the specific repository used for this assignment, following the principle of least privilege. The service was configured to use Docker as the runtime environment, deployed in the closest available region, and run on the free instance type. After completing the setup, Render automatically built the Docker image and deployed the application. Once the deployment finished successfully, I opened the generated public URL and confirmed that the API was running correctly by receiving a JSON response from the root endpoint..

Issue encountered: At first, the repository did not appear when selecting a source for the web service because GitHub permissions were not fully configured for Render.

Resolution: This was resolved by updating the GitHub App (Render) permissions to allow access to the selected repository. After this change, the repository became visible in Render and deployment proceeded successfully. Once deployment completed, the service URL was opened in the browser and returned a valid JSON welcome message, confirming that the application was running correctly in the cloud.

Assignment 2: PaaS with GitHub deployment (Full version)

Course: Cloud Computing & Data Engineering

By: Hamed Ahmadiania

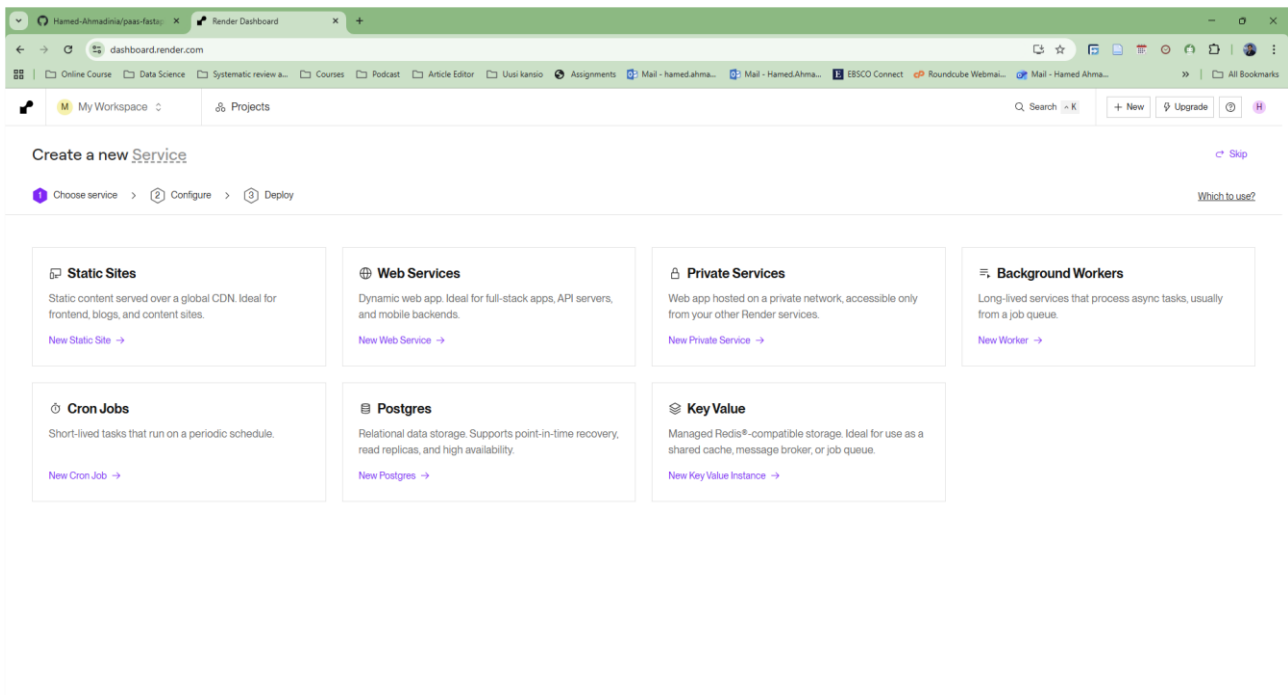


Figure 4 Render dashboard interface used to create a new Web Service for deploying the FastAPI application

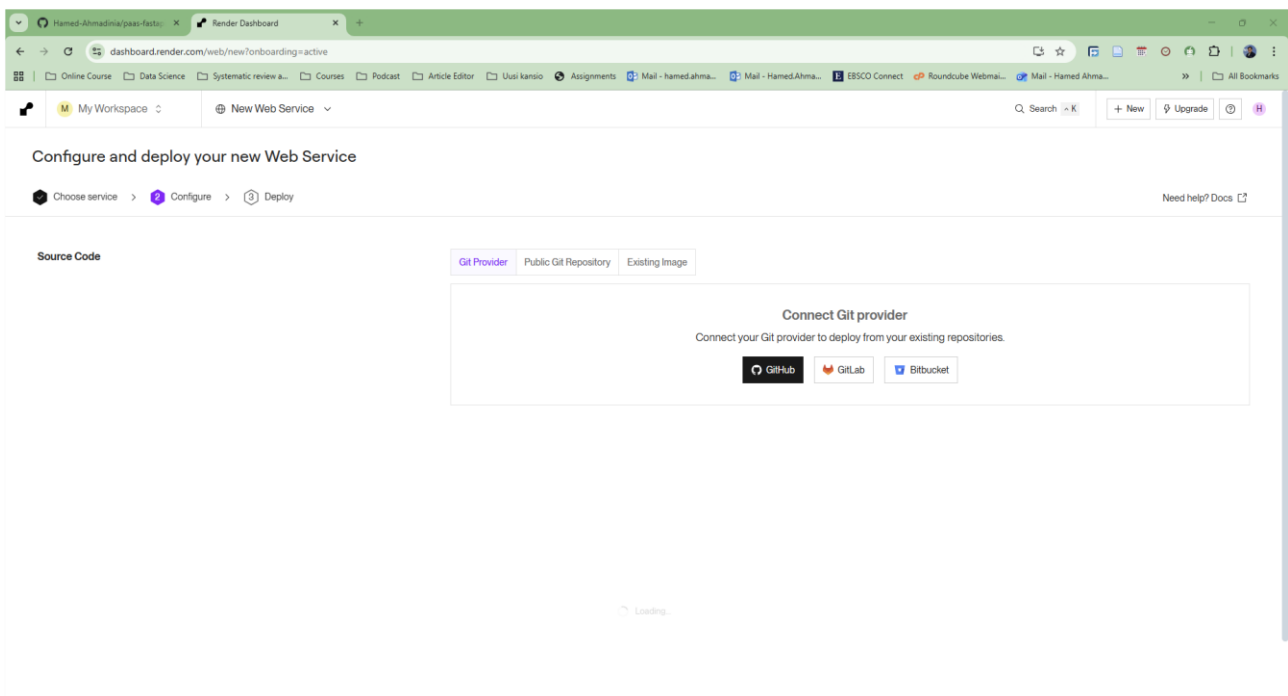


Figure 5 Connecting GitHub as the source repository when configuring a new Web Service on Render.

Assignment 2: PaaS with GitHub deployment (Full version)

Course: Cloud Computing & Data Engineering

By: Hamed Ahmadinia

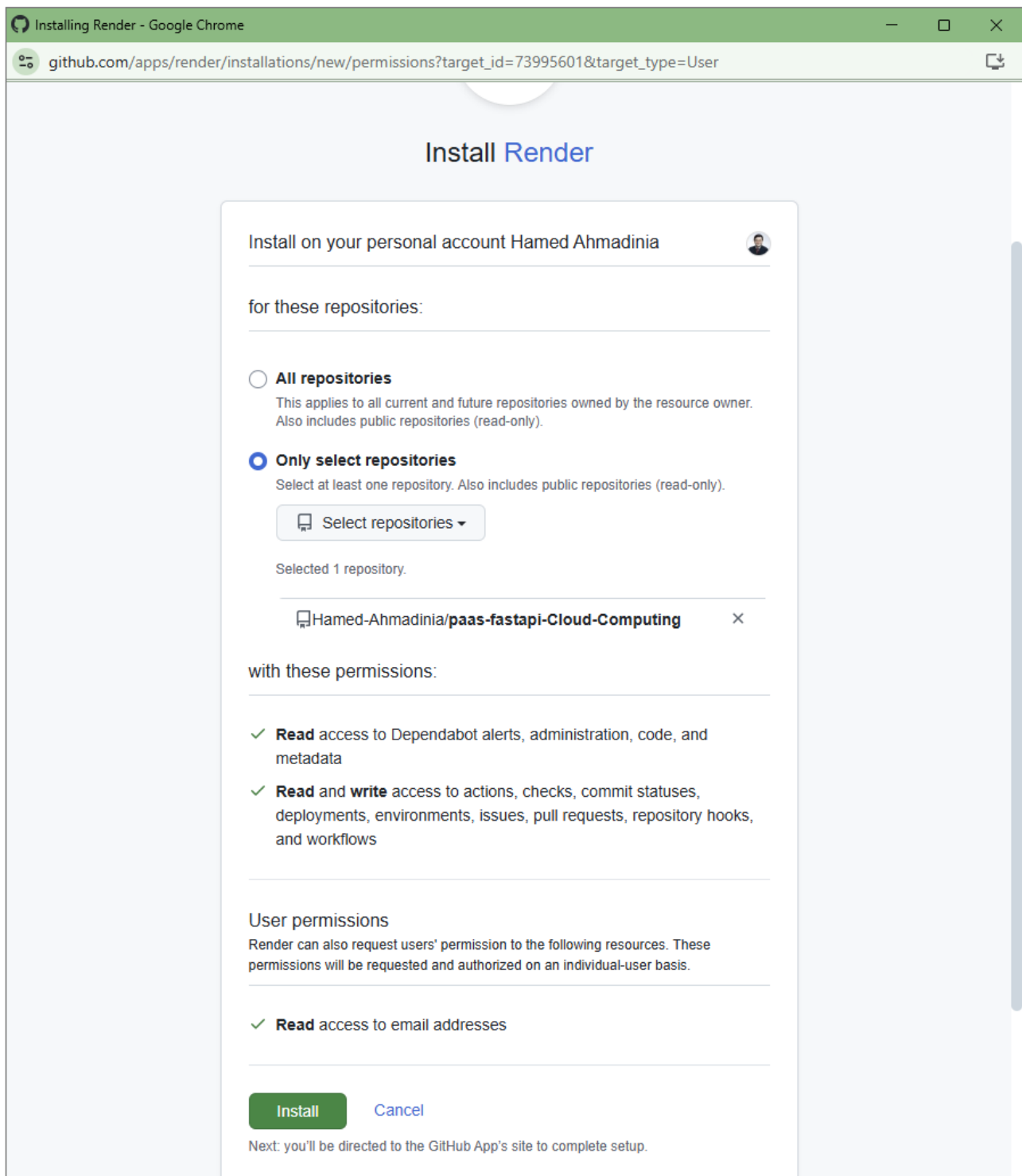


Figure 6 Granting Render permission to access only the project repository during GitHub integration.

Assignment 2: PaaS with GitHub deployment (Full version)

Course: Cloud Computing & Data Engineering

By: Hamed Ahmadinia

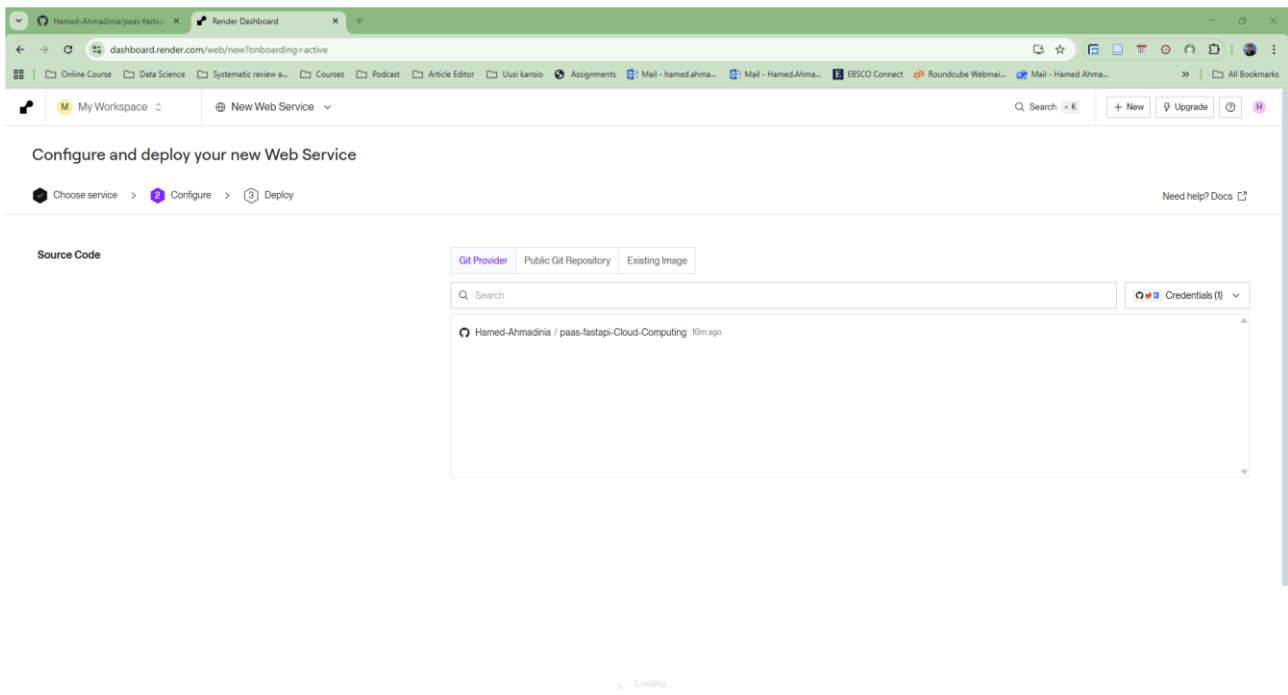


Figure 7 Render repository selection screen after GitHub permissions were correctly configured.

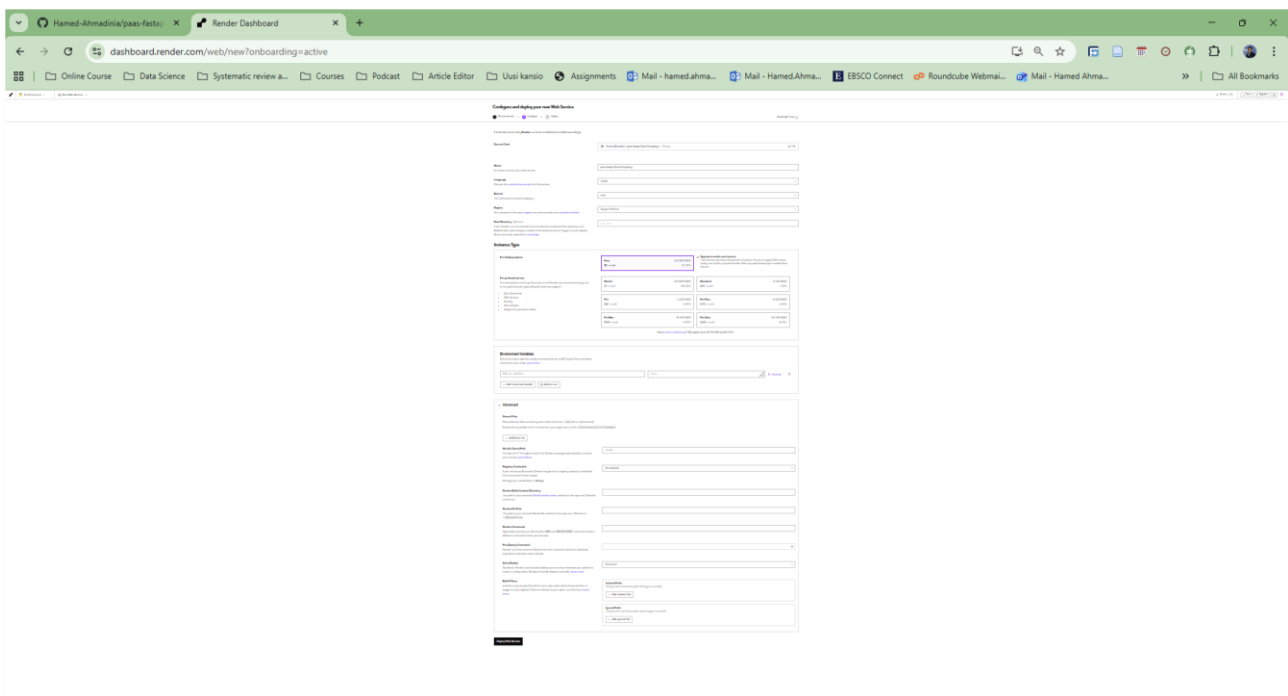


Figure 8 Render service setup confirming Docker as the runtime environment and free instance selection.

Assignment 2: PaaS with GitHub deployment (Full version)

Course: Cloud Computing & Data Engineering

By: Hamed Ahmadiania

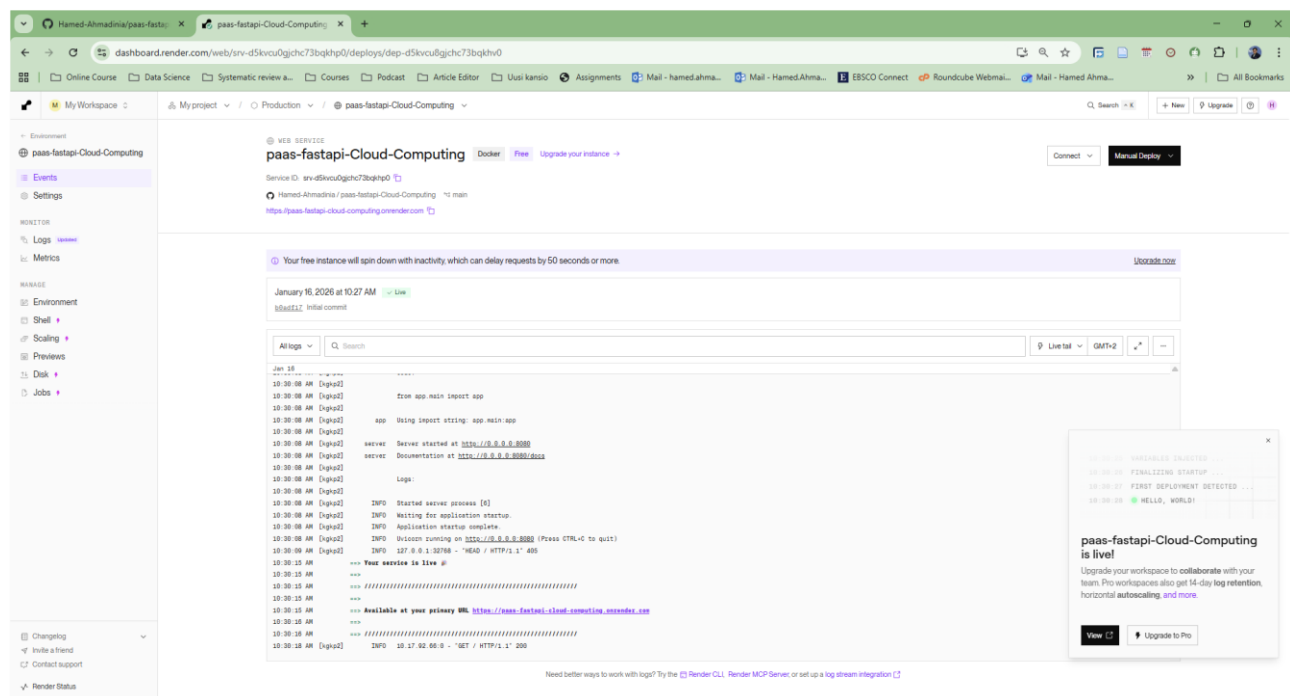


Figure 9 Successful deployment logs on Render confirming that the Docker-based FastAPI service is live and running.

Part C – Cloning the Repository Locally Using SSH: In this part, I set up SSH access to GitHub and cloned the project repository to my local computer. First, I verified that Git was installed and working correctly. I then checked that my SSH keys were available locally and added the public key to my GitHub account under SSH and GPG key settings. To confirm that the SSH connection was working, I tested the connection using the `ssh -T git@github.com` command, which successfully authenticated my account. After that, I copied the SSH repository URL from GitHub and cloned the repository to my local machine using Git. Finally, I opened the cloned project folder in Visual Studio Code to continue development locally.

Issue encountered: Initially, SSH authentication failed because the SSH key was not correctly linked to GitHub.

Resolution: This was fixed by adding the correct public SSH key to GitHub and configuring the SSH client. After this, SSH authentication worked and the repository was cloned successfully.

Assignment 2: PaaS with GitHub deployment (Full version)

Course: Cloud Computing & Data Engineering

By: Hamed Ahmadiania

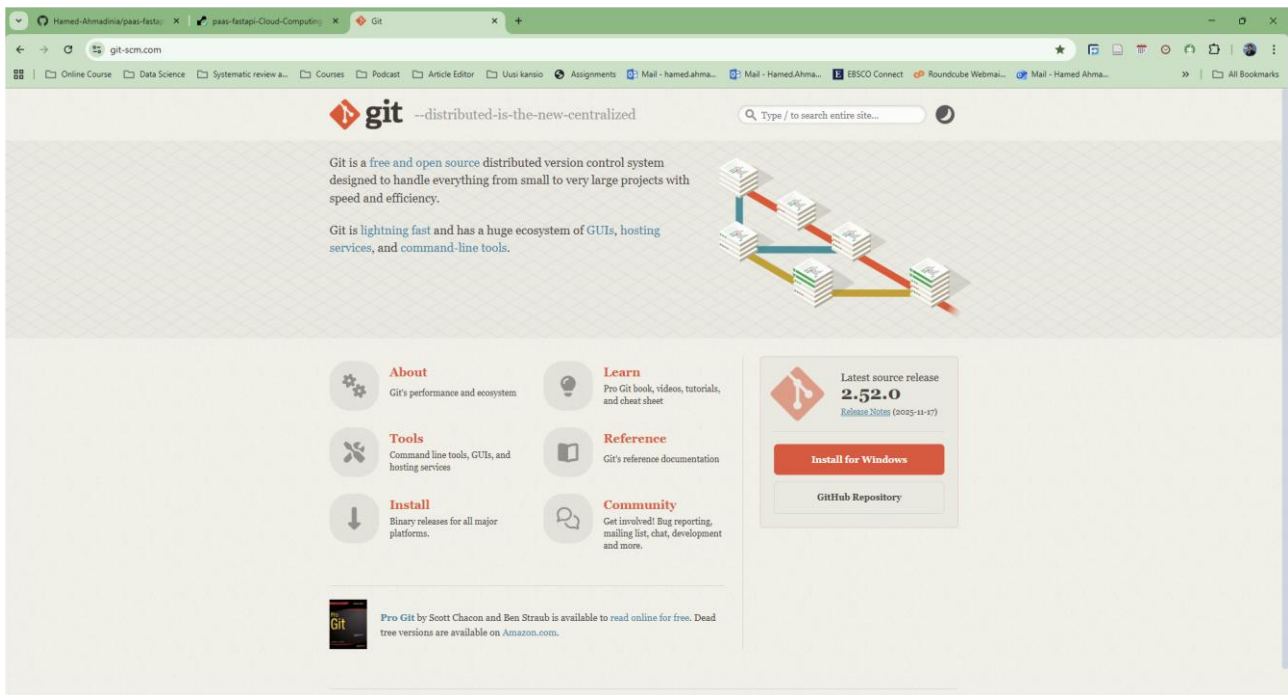


Figure 10 Git installed on the local machine to enable repository cloning and version control.

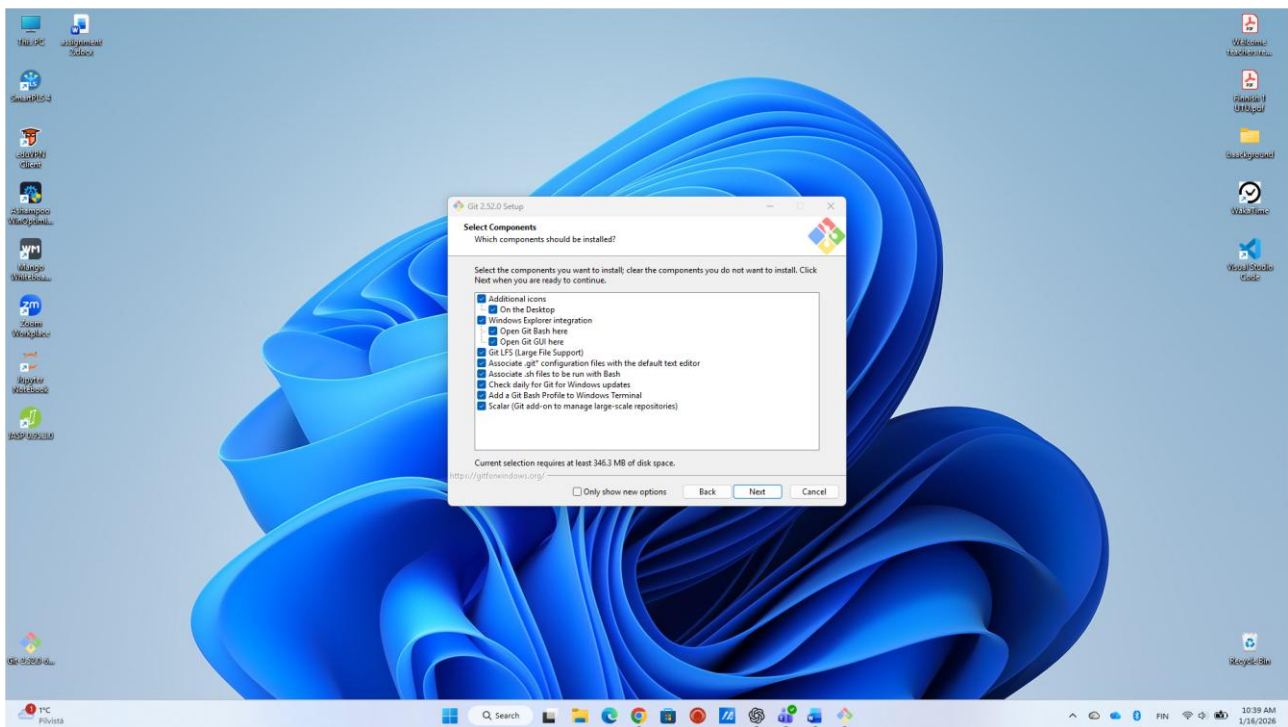


Figure 11 Official Git website used to install Git on the local system - Step 1

Assignment 2: PaaS with GitHub deployment (Full version)

Course: Cloud Computing & Data Engineering

By: Hamed Ahmadinia

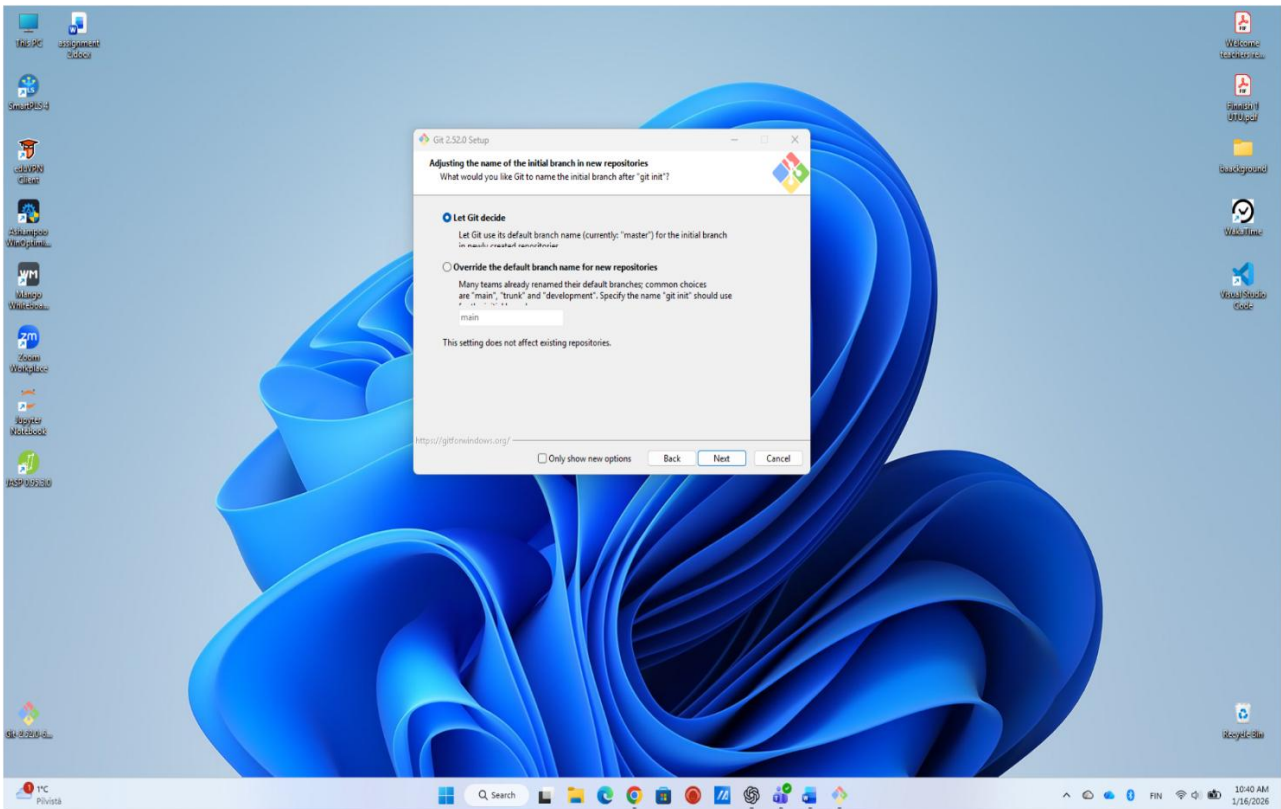


Figure 12 Official Git website used to install Git on the local system - Step 2

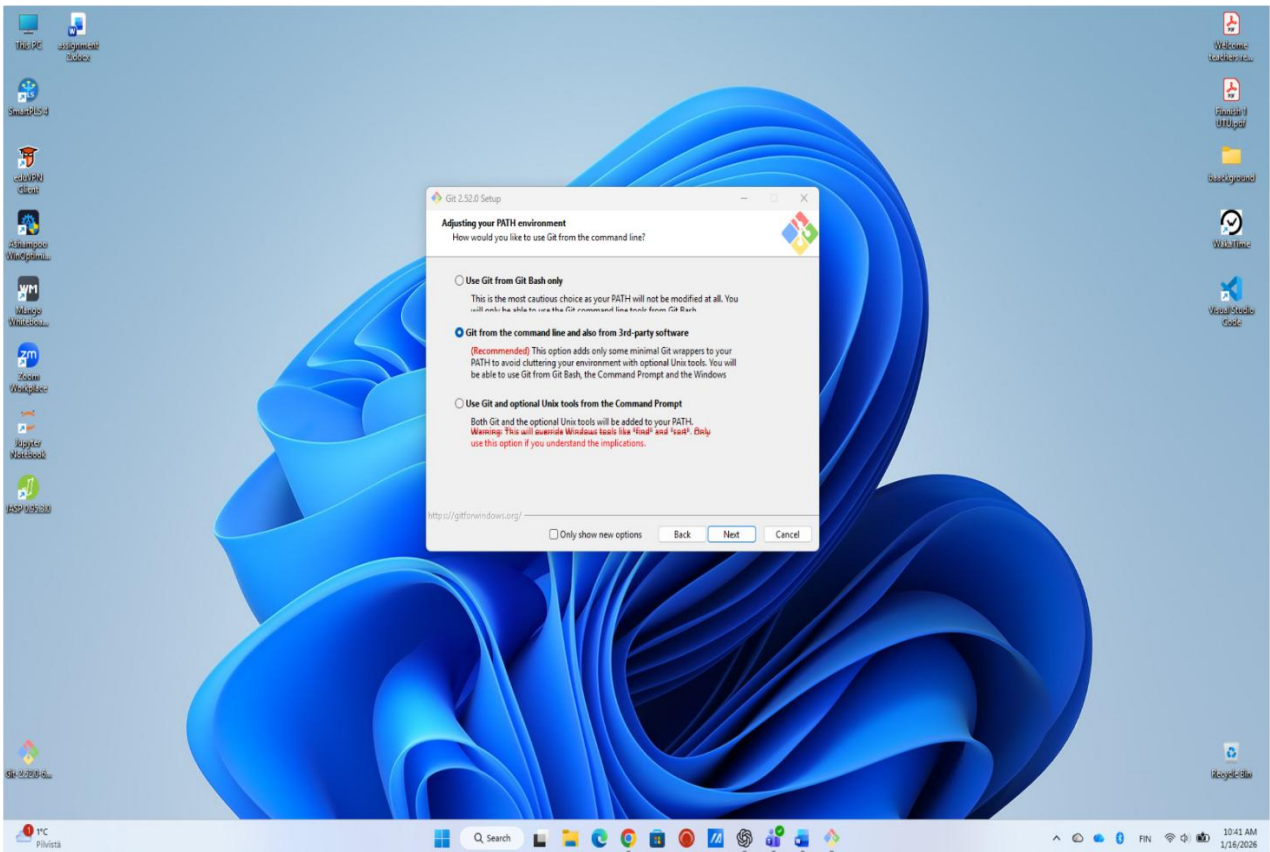


Figure 13 Official Git website used to install Git on the local system - Step 3

Assignment 2: PaaS with GitHub deployment (Full version)

Course: Cloud Computing & Data Engineering

By: Hamed Ahmadiania

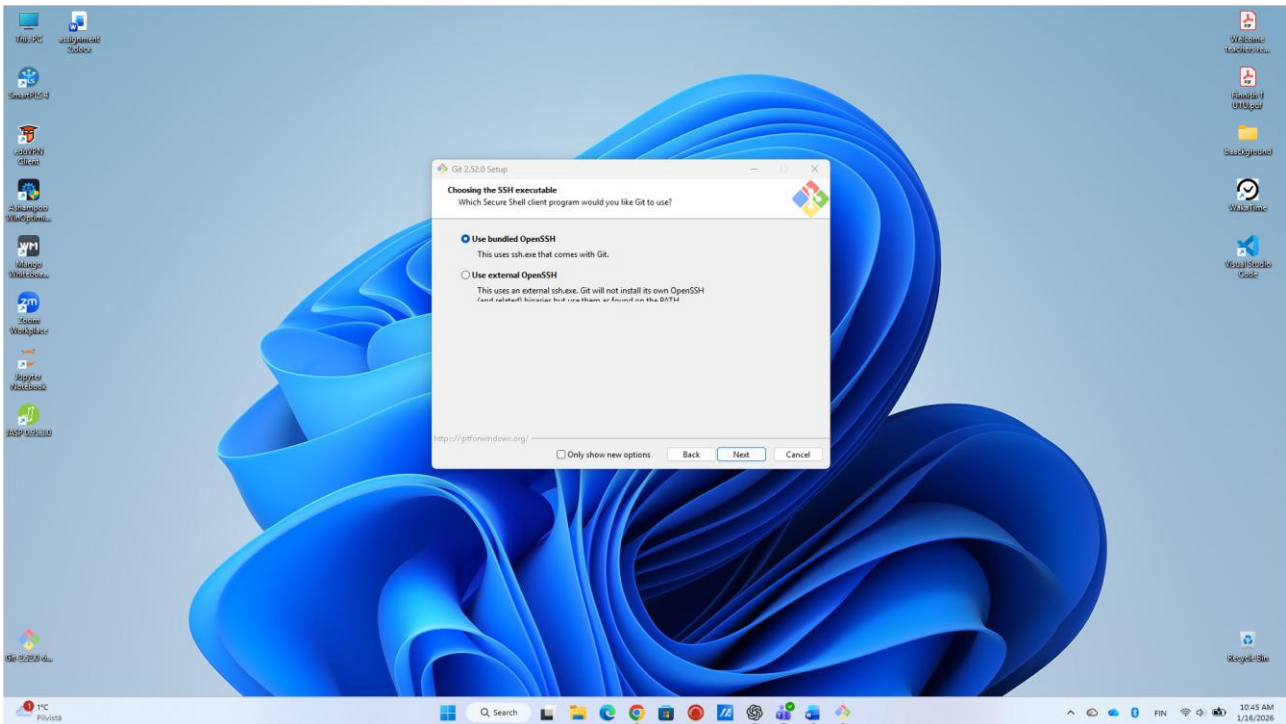


Figure 14 Official Git website used to install Git on the local system - Step 4

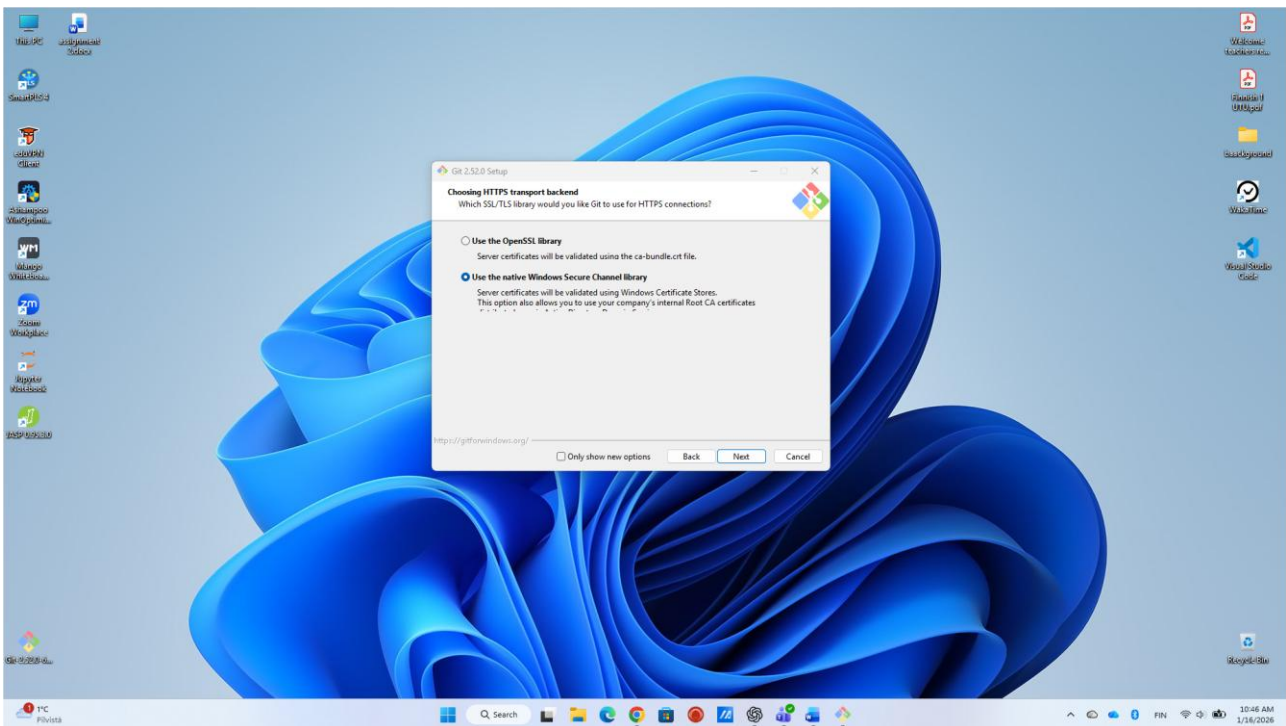


Figure 15 Official Git website used to install Git on the local system - Step 5

Assignment 2: PaaS with GitHub deployment (Full version)

Course: Cloud Computing & Data Engineering

By: Hamed Ahmadinia

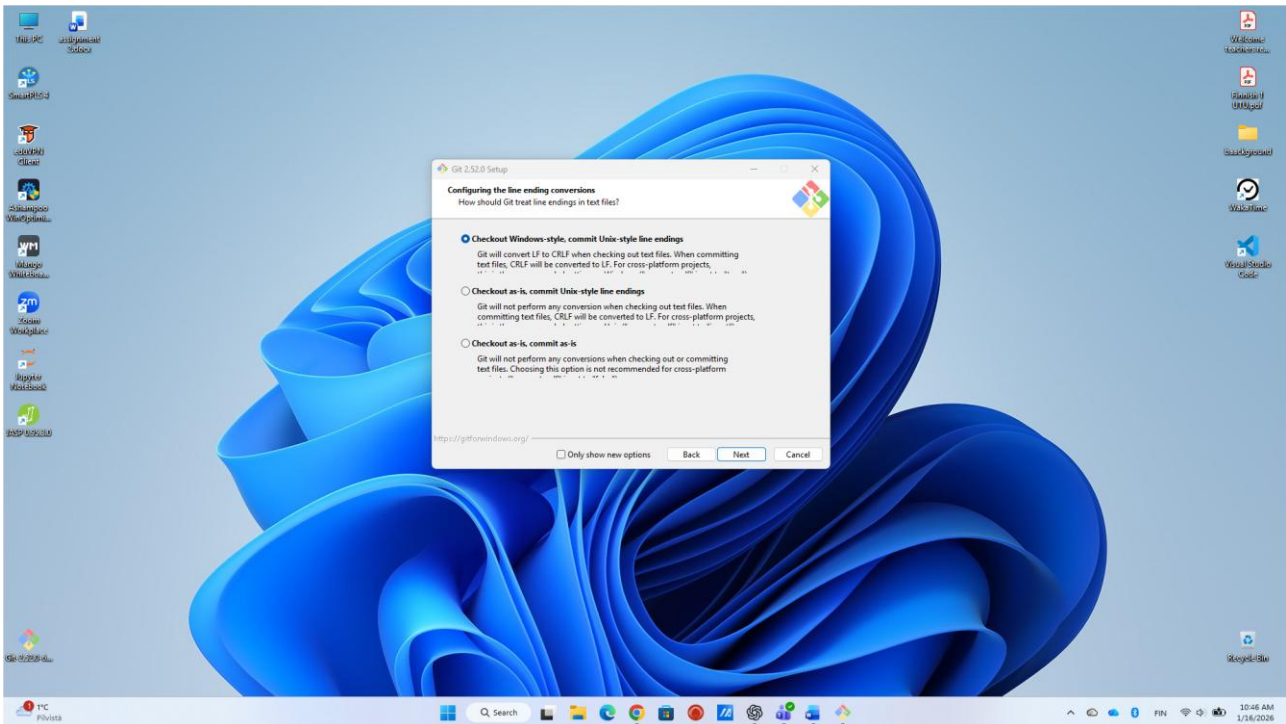


Figure 16 Official Git website used to install Git on the local system - Step 6

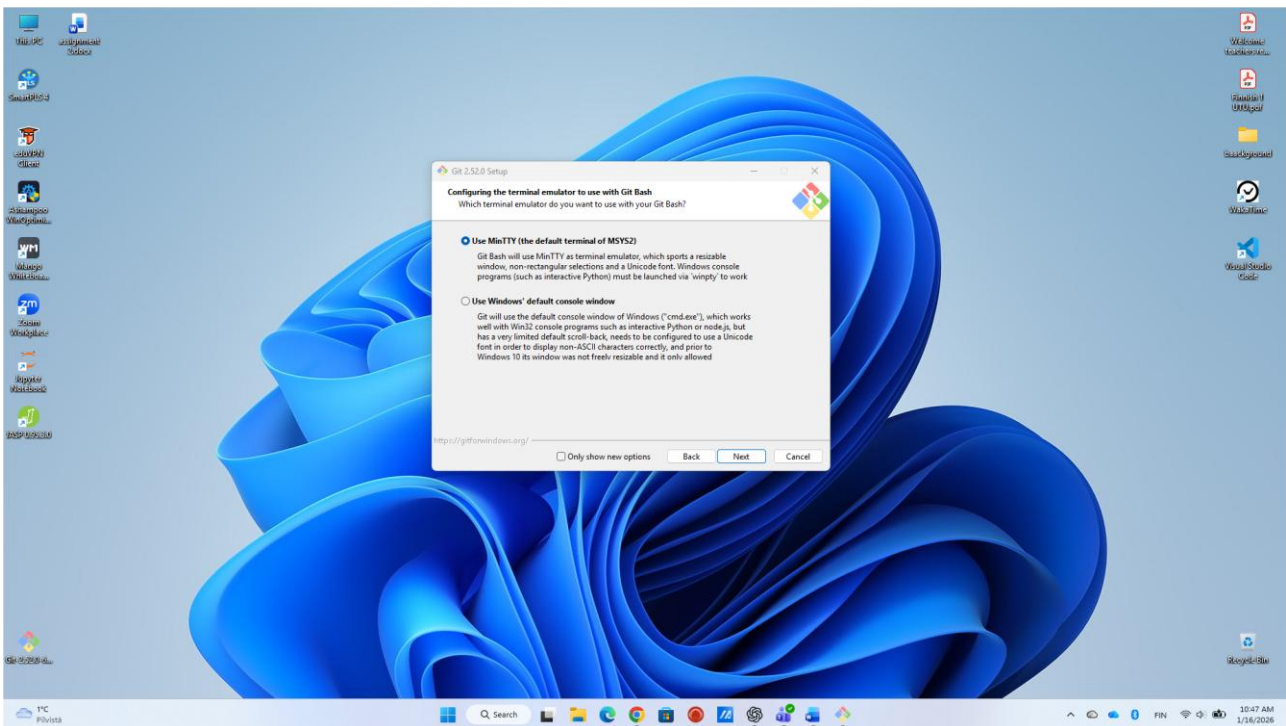


Figure 17 Official Git website used to install Git on the local system - Step 7

Assignment 2: PaaS with GitHub deployment (Full version)

Course: Cloud Computing & Data Engineering

By: Hamed Ahmadiania

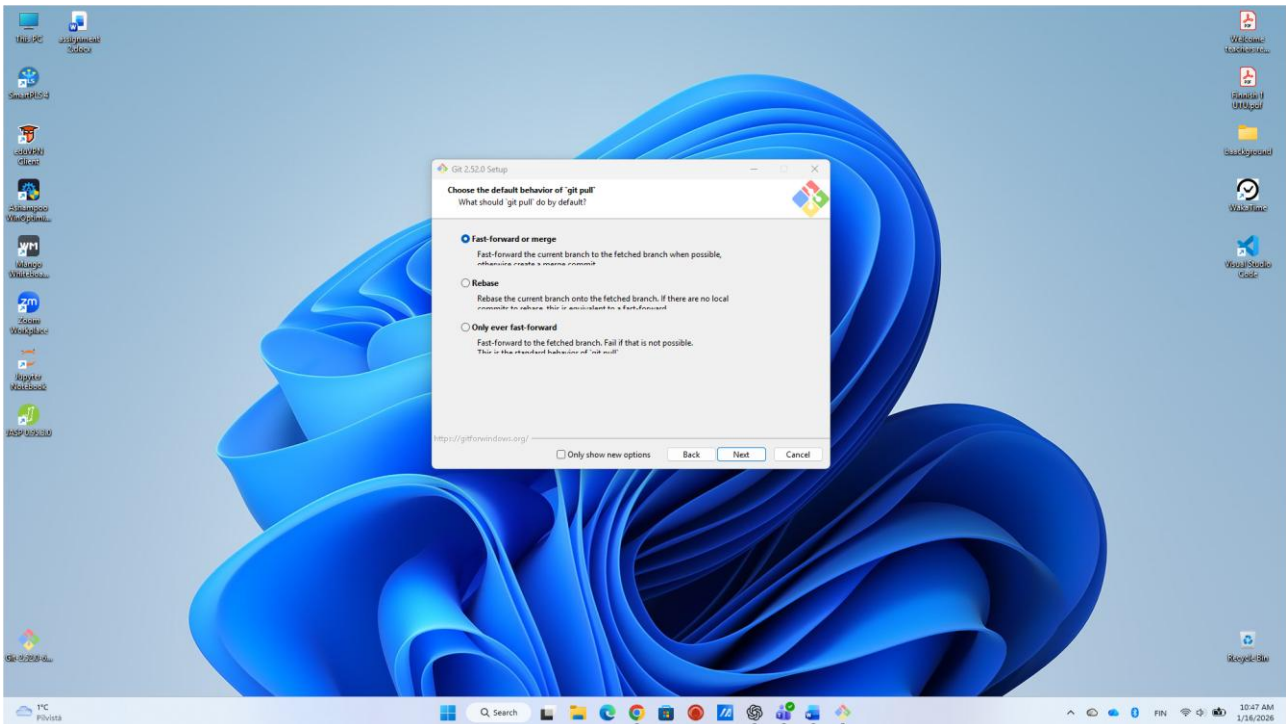


Figure 18 Official Git website used to install Git on the local system - Step 8

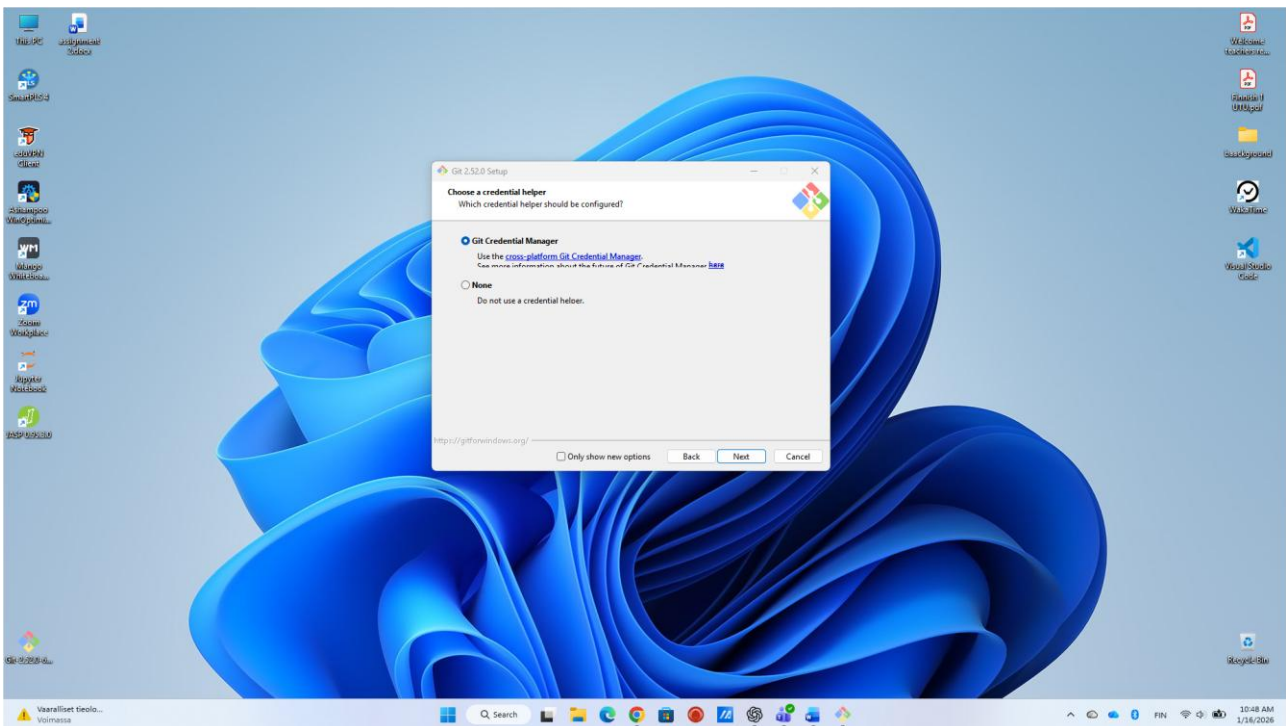


Figure 19 Official Git website used to install Git on the local system - Step 9

Assignment 2: PaaS with GitHub deployment (Full version)

Course: Cloud Computing & Data Engineering

By: Hamed Ahmadinia

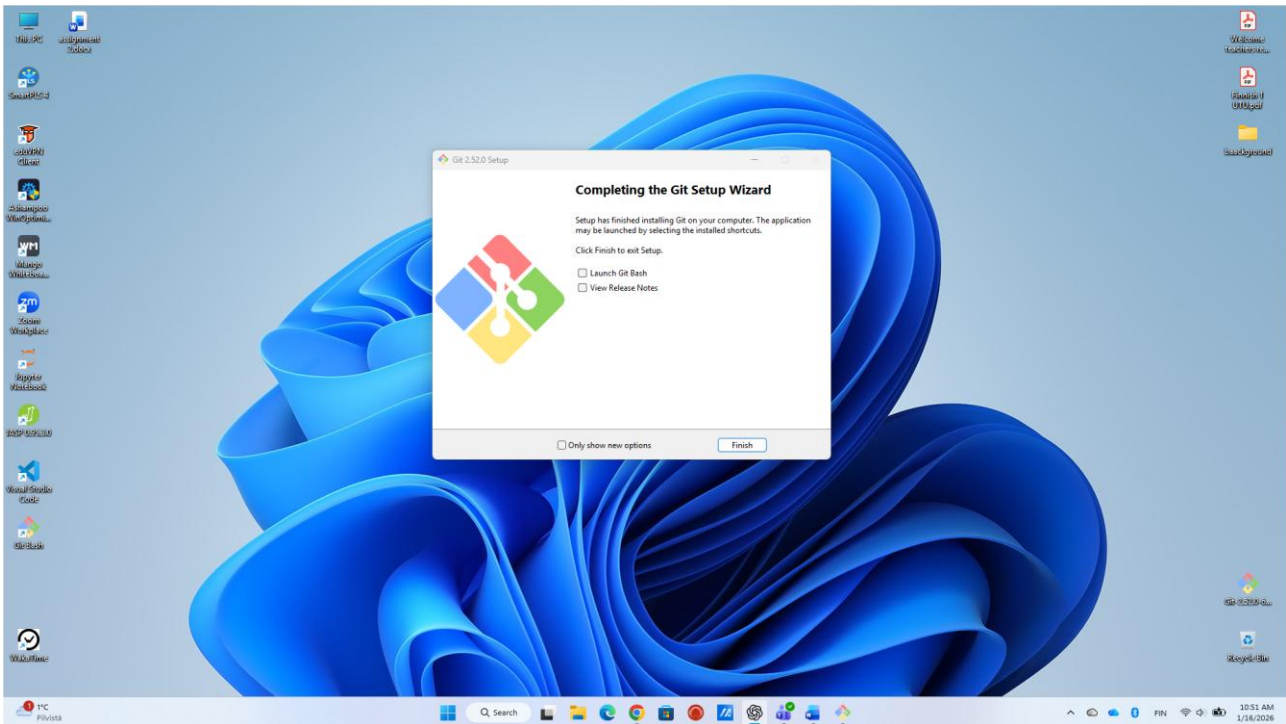


Figure 20 Git installation completed successfully on the local machine.

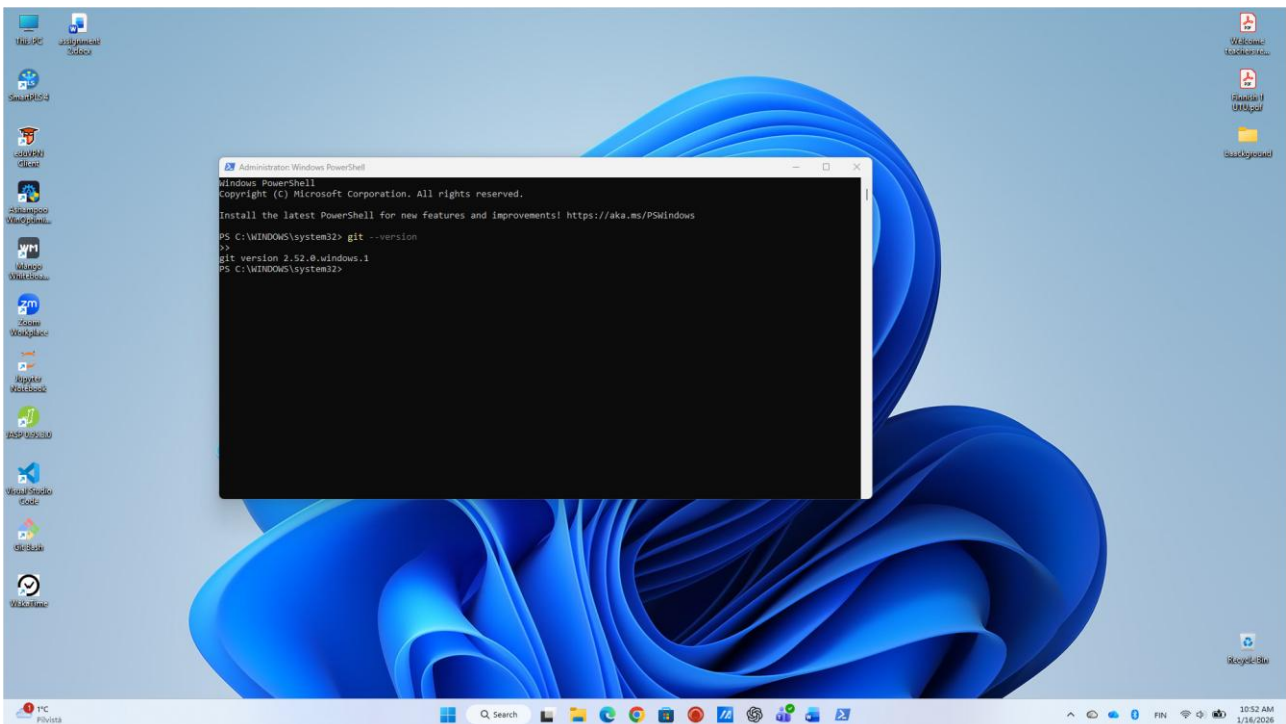


Figure 21 Verifying Git installation locally using PowerShell (git --version).

Assignment 2: PaaS with GitHub deployment (Full version)

Course: Cloud Computing & Data Engineering

By: Hamed Ahmadiania

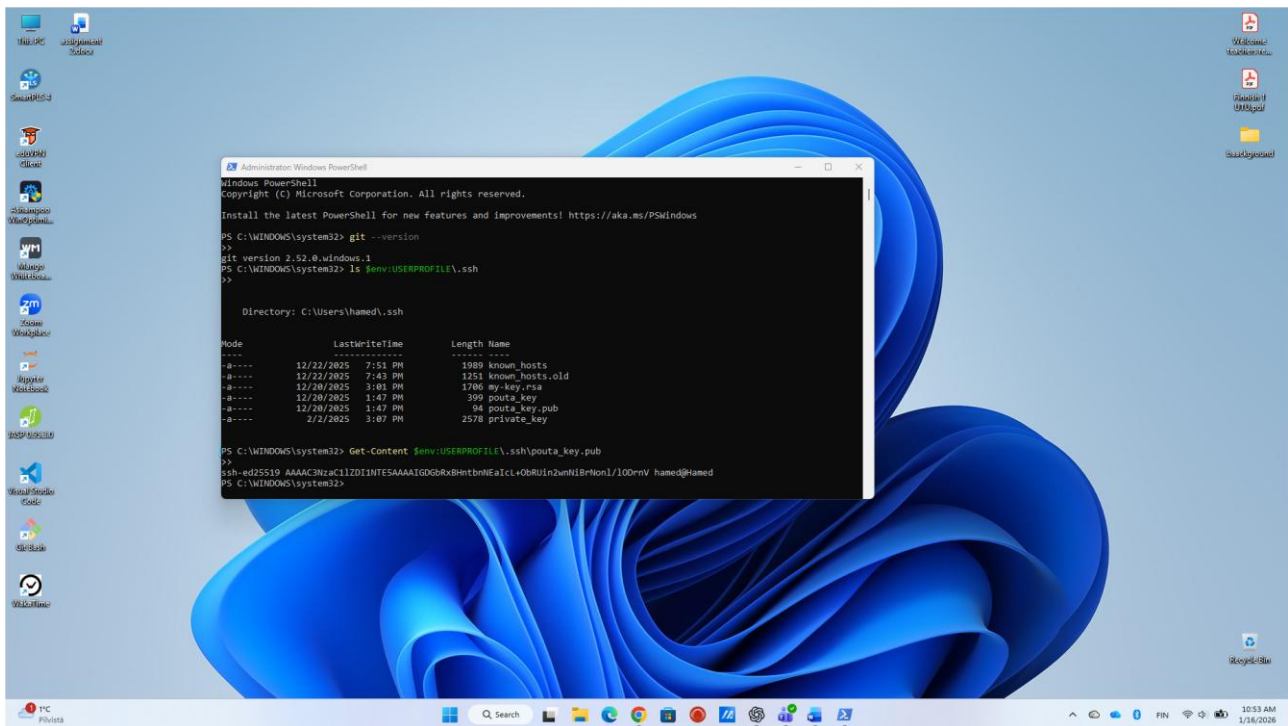


Figure 22 Verifying existing SSH keys and displaying the public SSH key for GitHub configuration

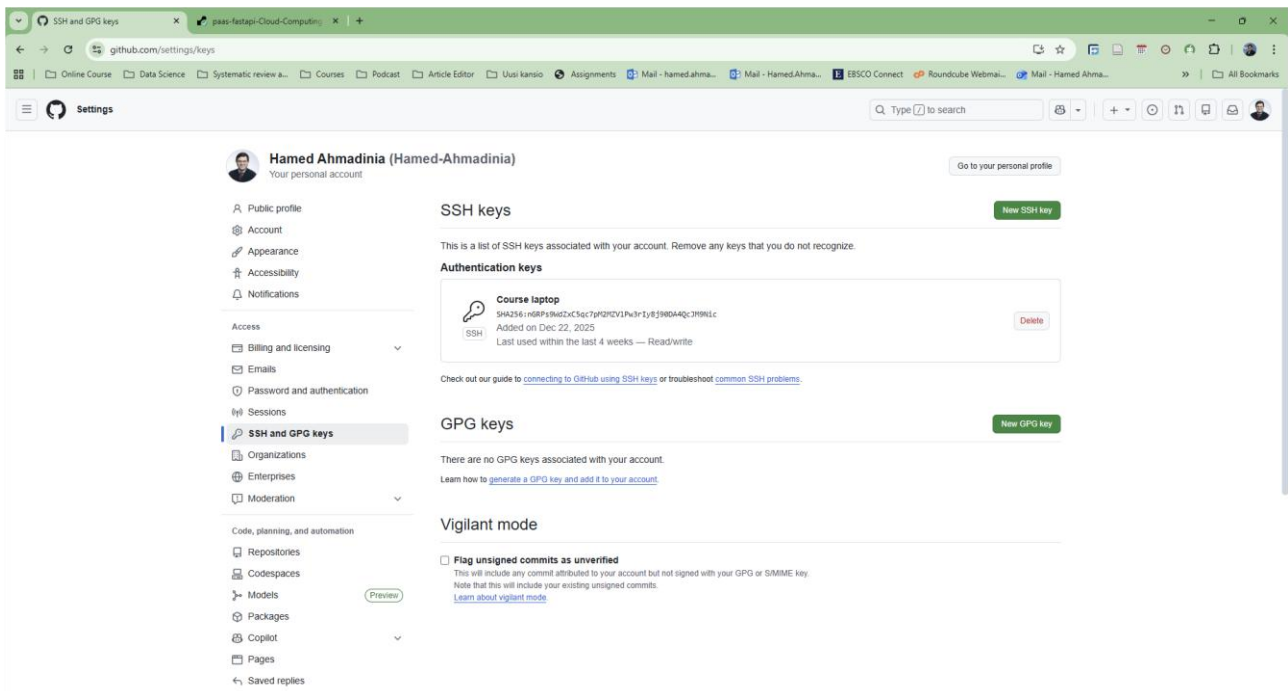


Figure 23 SSH public key successfully added to GitHub for secure repository access.

Assignment 2: PaaS with GitHub deployment (Full version)

Course: Cloud Computing & Data Engineering

By: Hamed Ahmadiania

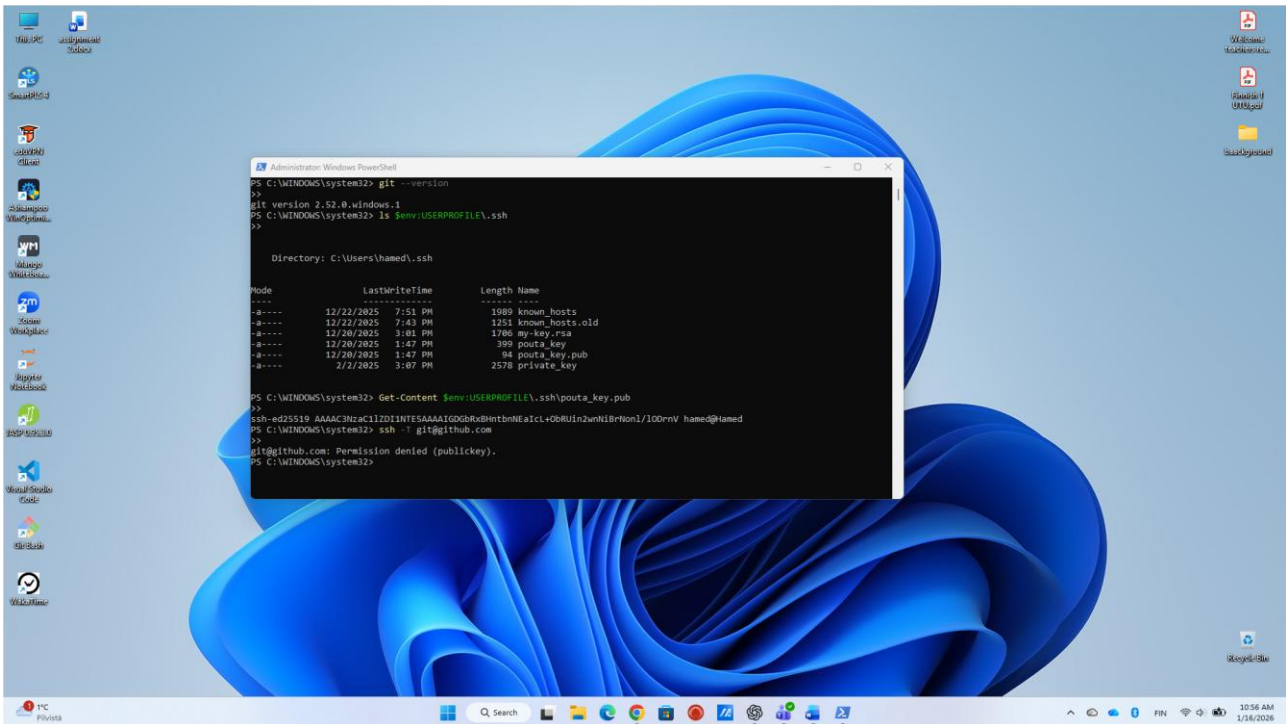


Figure 24 Verifying SSH key setup and successful authentication with GitHub via PowerShell.

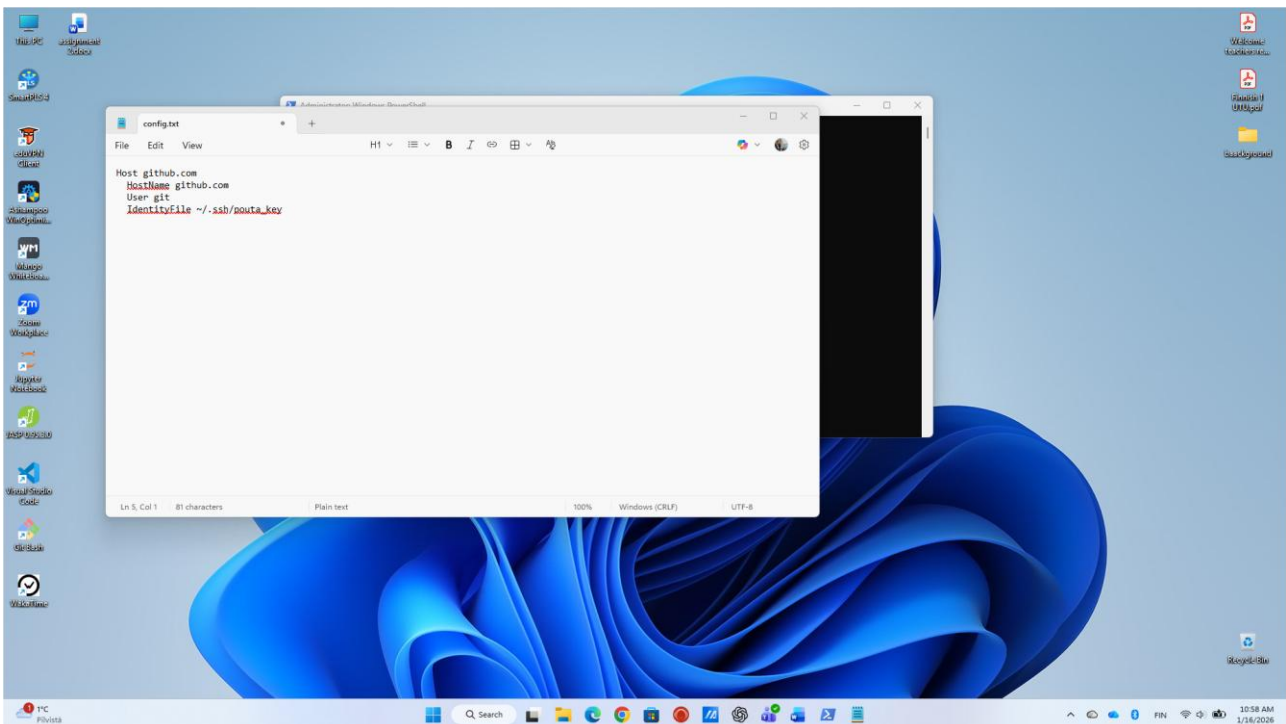


Figure 25 SSH configuration file specifying the GitHub host and identity key used for authentication.

Assignment 2: PaaS with GitHub deployment (Full version)

Course: Cloud Computing & Data Engineering

By: Hamed Ahmadiania

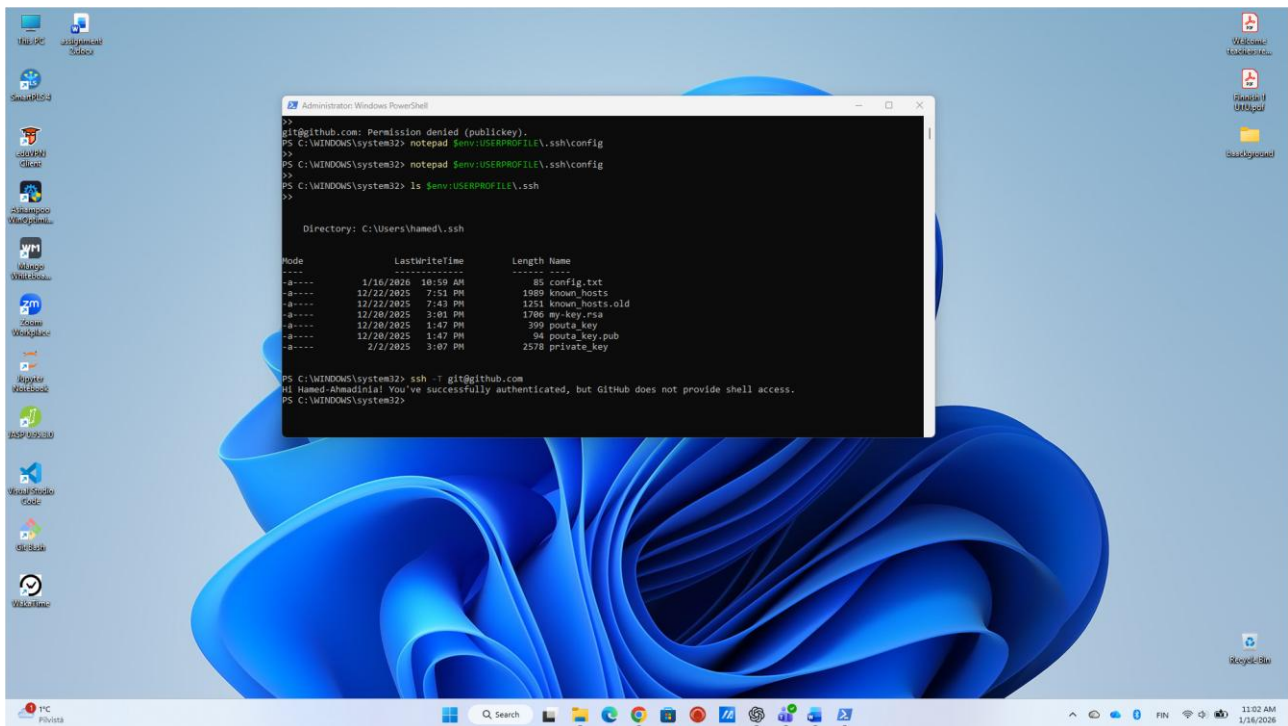


Figure 26 Successful SSH authentication test with GitHub confirming SSH access is correctly configured.

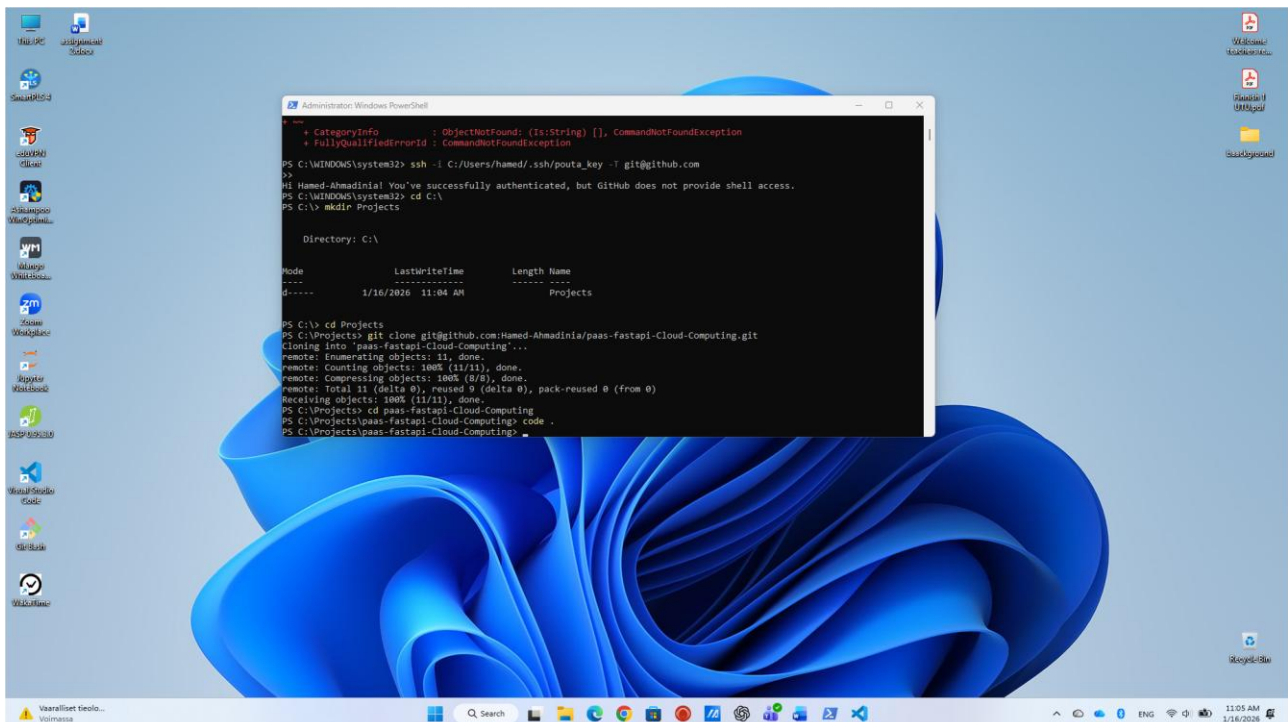


Figure 27 Successful SSH authentication, cloning the GitHub repository via SSH, and opening the project in Visual Studio Code

Assignment 2: PaaS with GitHub deployment (Full version)

Course: Cloud Computing & Data Engineering

By: Hamed Ahmadinia

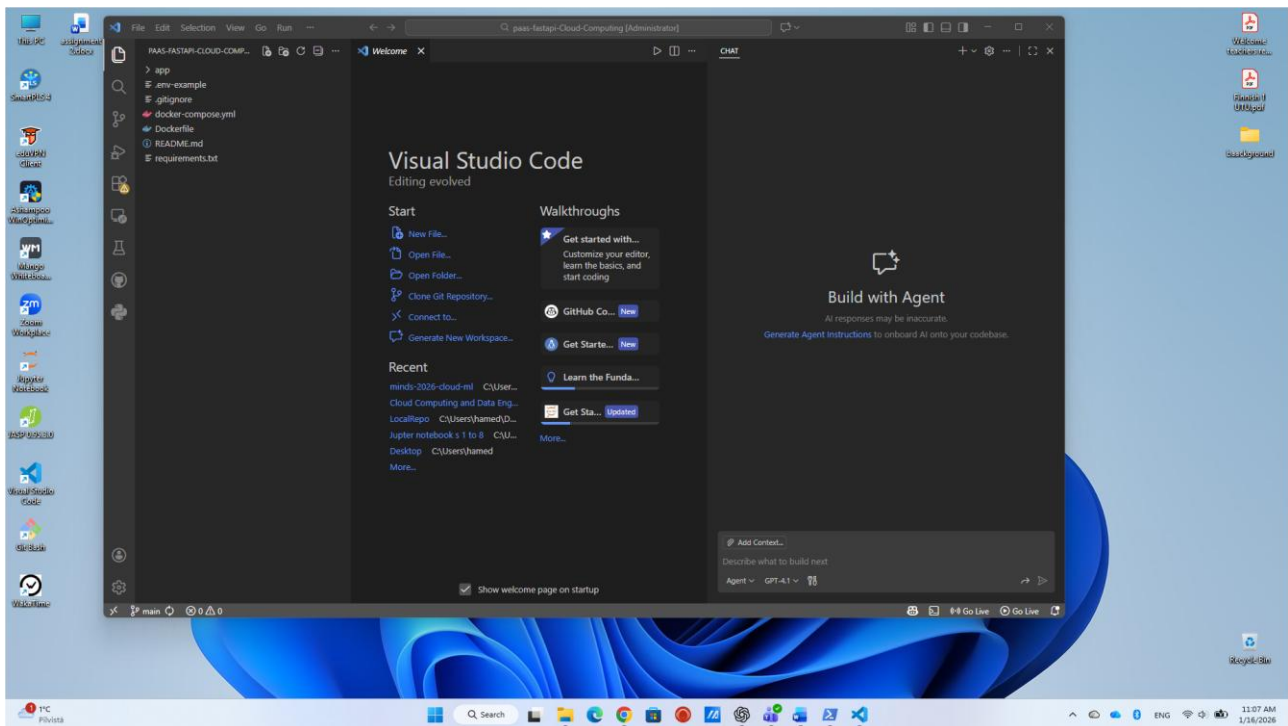


Figure 28 Project repository opened locally in Visual Studio Code after cloning via SSH

Part D – Local Code Changes and Testing with Docker Compose: In this step, I modified the FastAPI application locally and ran it using Docker Compose. I opened the project in Visual Studio Code and edited the `app/main.py` file to change the default Hello message returned by the root endpoint. This confirmed that I understood how to safely modify FastAPI routes while keeping the JSON structure correct. After making the code change, I used Docker Compose to build and run the application locally. The service was started using `docker-compose up --build`, and the application was successfully exposed on the configured port. When needed, existing containers were stopped using `docker-compose down` before rebuilding. The container logs confirmed that the application started correctly, and the API responded as expected in the browser. This step verified that local code changes were correctly reflected when running the application inside Docker containers..

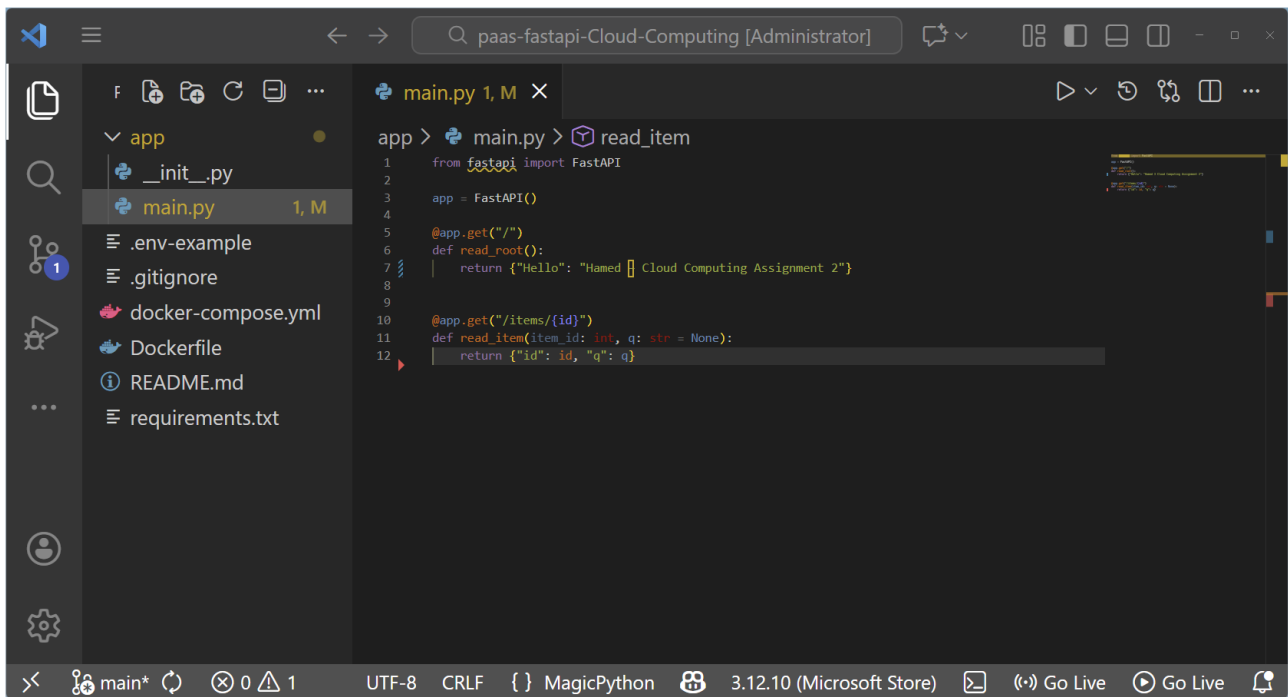
Issue encountered: During local testing, existing Docker containers needed to be stopped before rebuilding the service.

Resolution: This was resolved by running `docker-compose down` before rebuilding the application with `docker-compose up --build`. After this, the service started correctly and the API responded as expected.

Assignment 2: PaaS with GitHub deployment (Full version)

Course: Cloud Computing & Data Engineering

By: Hamed Ahmadiania



```
main.py 1, M X
app > main.py > read_item
1 from fastapi import FastAPI
2
3 app = FastAPI()
4
5 @app.get("/")
6 def read_root():
7     return {"Hello": "Hamed Cloud Computing Assignment 2"}
8
9
10 @app.get("/items/{id}")
11 def read_item(item_id: int, q: str = None):
12     return {"id": id, "q": q}
```

Figure 29 Edited app/main.py in VS Code showing the updated welcome message for the FastAPI root endpoint.

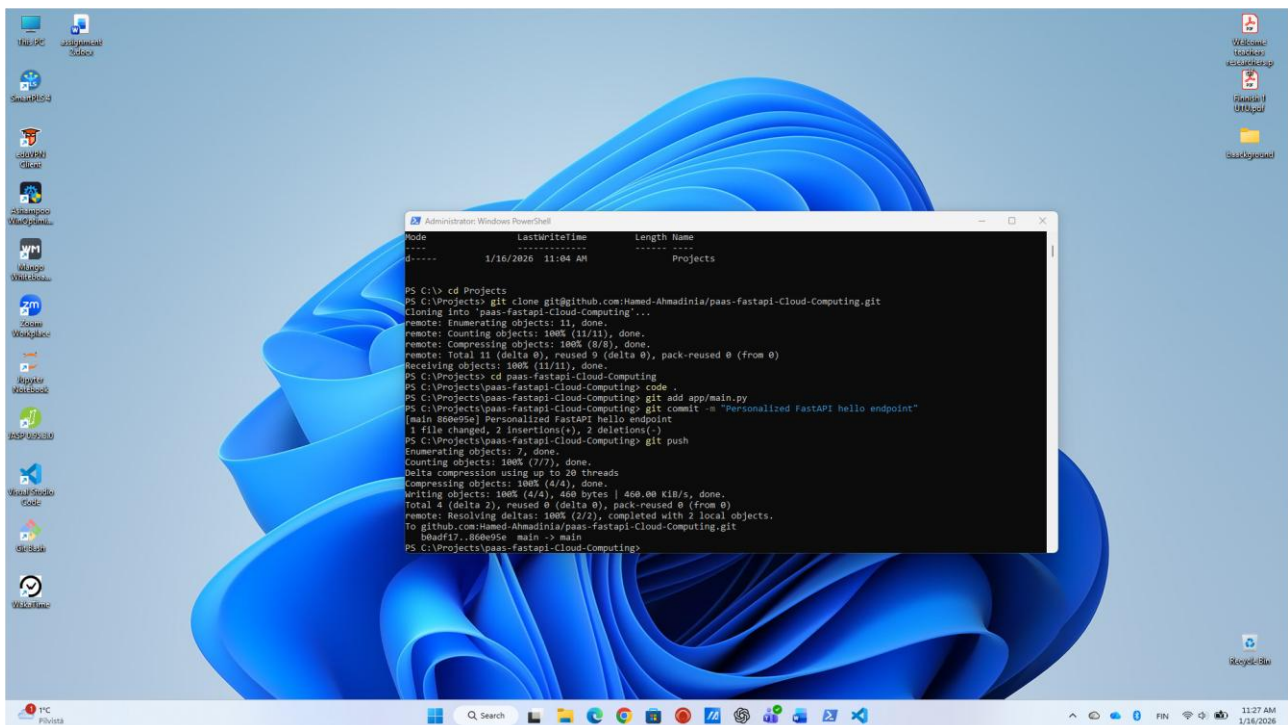


Figure 30 Git commit and push of the modified app/main.py, triggering automatic redeployment on Render.

Assignment 2: PaaS with GitHub deployment (Full version)

Course: Cloud Computing & Data Engineering

By: Hamed Ahmadiania

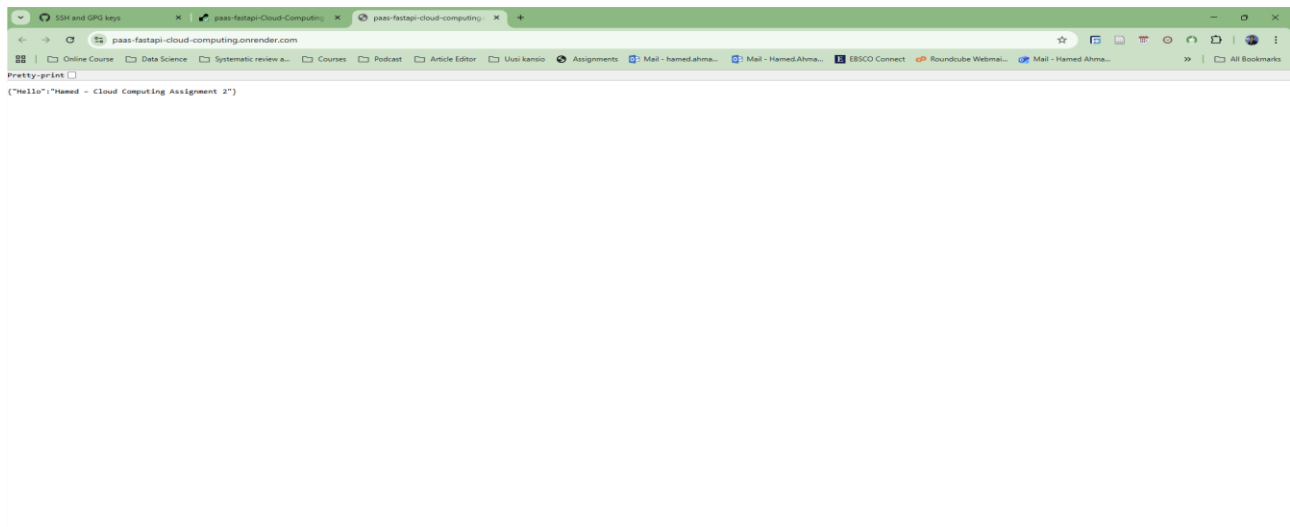


Figure 31 Live FastAPI application deployed on Render, showing the updated JSON welcome message.

Part E – Local Testing with Docker (Recommended): In Part E, the FastAPI application was tested locally using Docker and Docker Compose. Docker Desktop was installed on the local machine and verified to be running correctly. From the project directory, the application was built and started using the command `docker-compose up --build`. Once the containers were running, the application was accessed in a web browser at `http://localhost:8000`, where the updated Hello message was displayed. The automatically generated API documentation was also tested using `http://localhost:8000/docs`. This confirmed that the application worked correctly in a local containerized environment before cloud deployment.

Issue encountered: No major issues were encountered during local Docker testing.

Resolution: The application started successfully, and the expected API responses were returned, confirming correct local container execution.

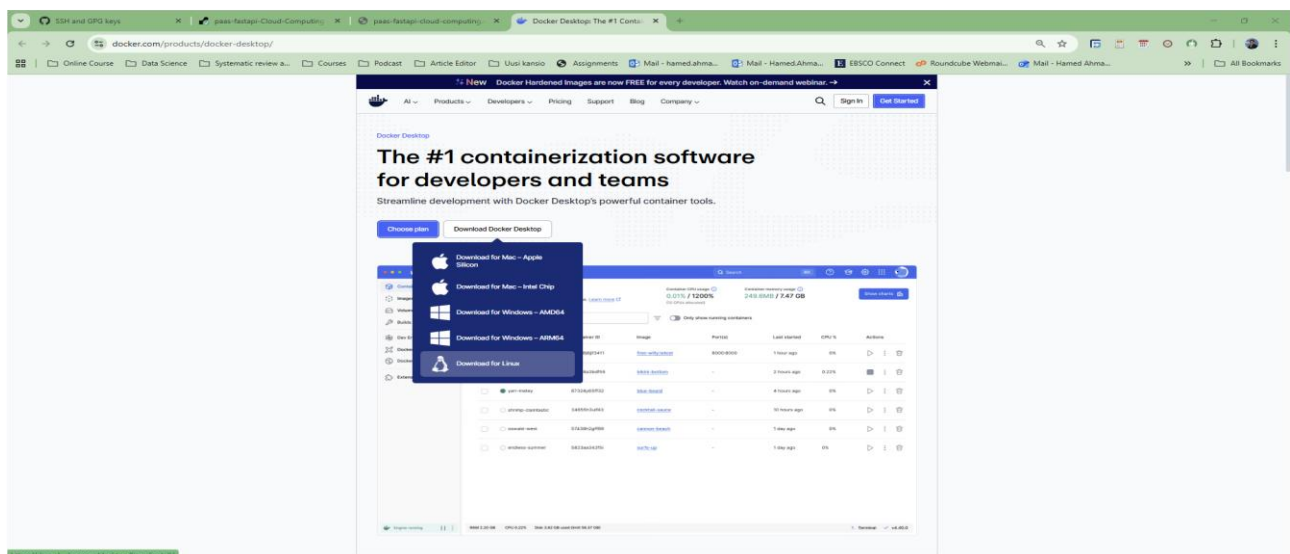


Figure 32 Installing Docker Desktop on Windows as preparation for local FastAPI testing with Docker Compose.

Assignment 2: PaaS with GitHub deployment (Full version)
Course: Cloud Computing & Data Engineering
By: Hamed Ahmadinia

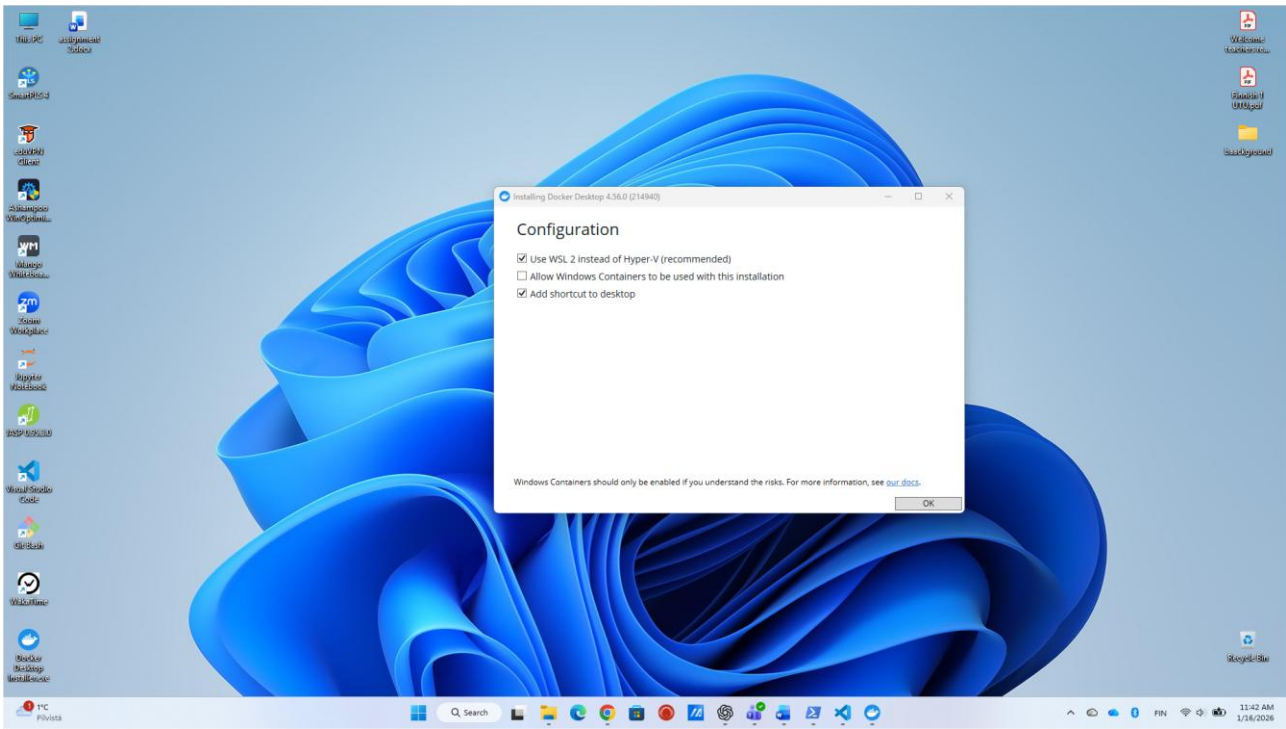


Figure 33 Docker Desktop installation configuration on Windows, enabling WSL 2 for local container execution.

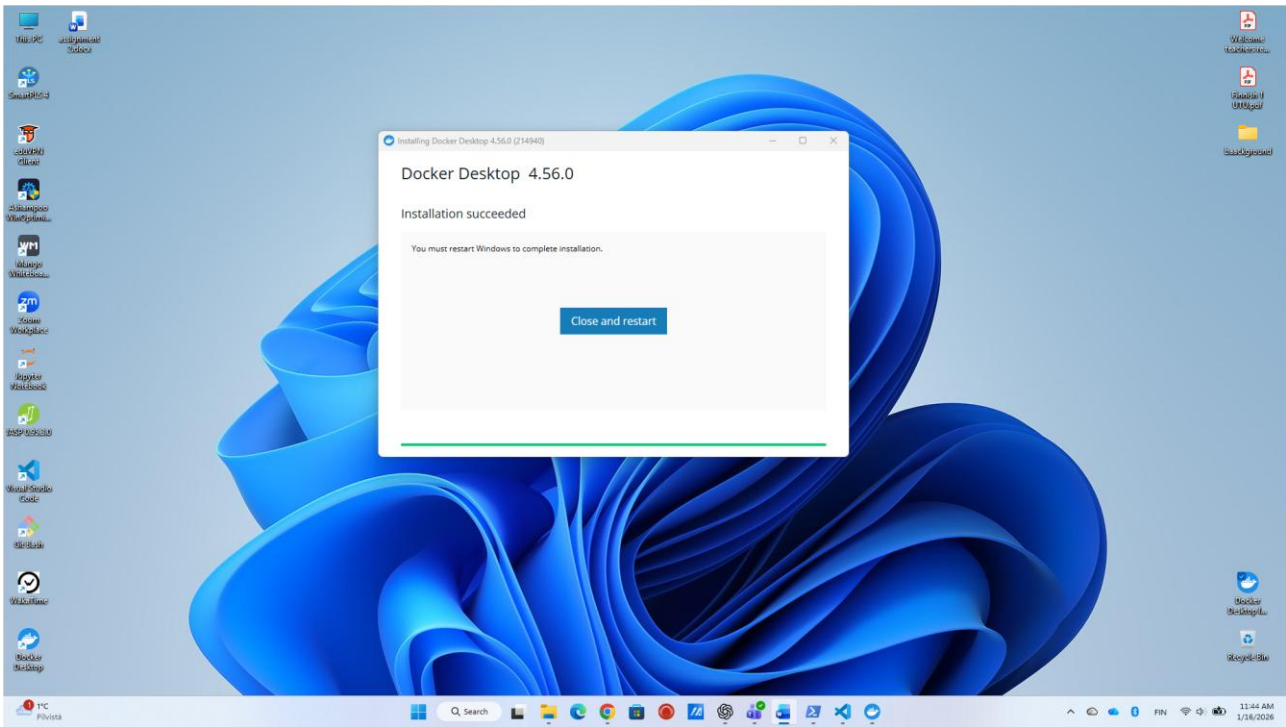


Figure 34 Successful completion of Docker Desktop installation on Windows

Assignment 2: PaaS with GitHub deployment (Full version)

Course: Cloud Computing & Data Engineering

By: Hamed Ahmadinia

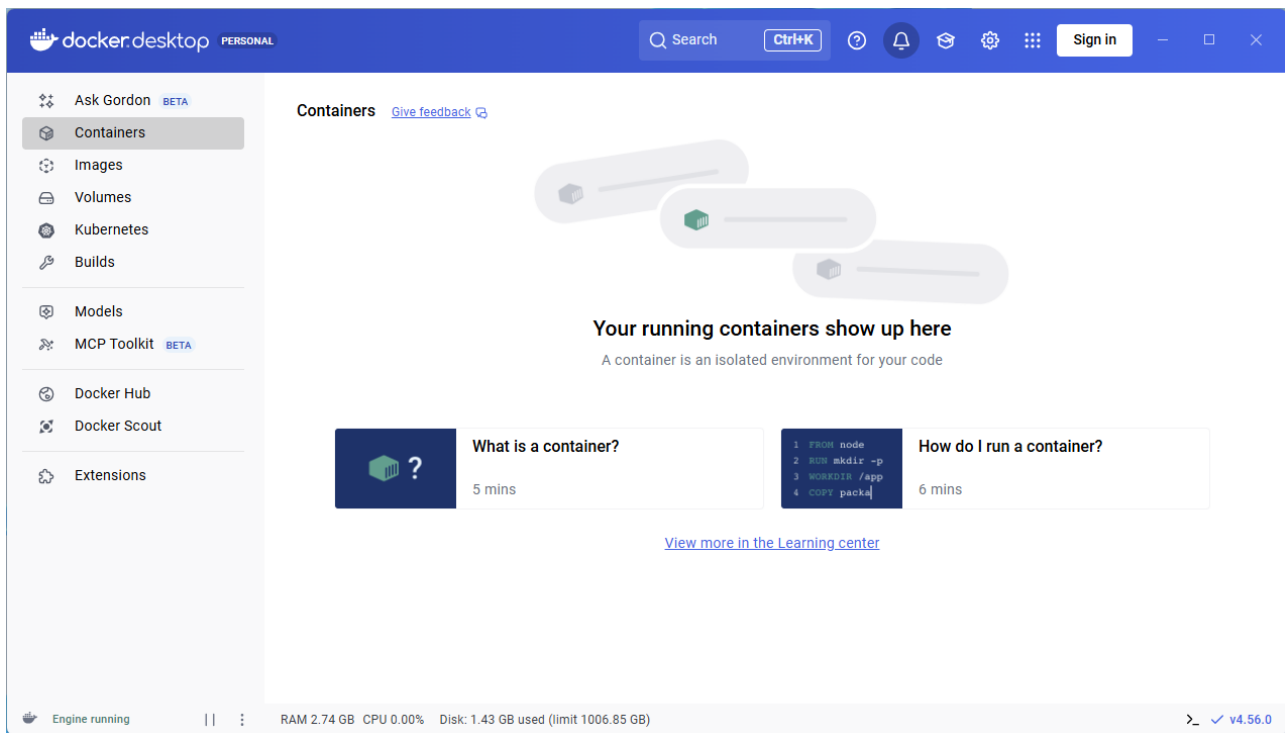


Figure 35 Docker Desktop running successfully with the Docker engine active.

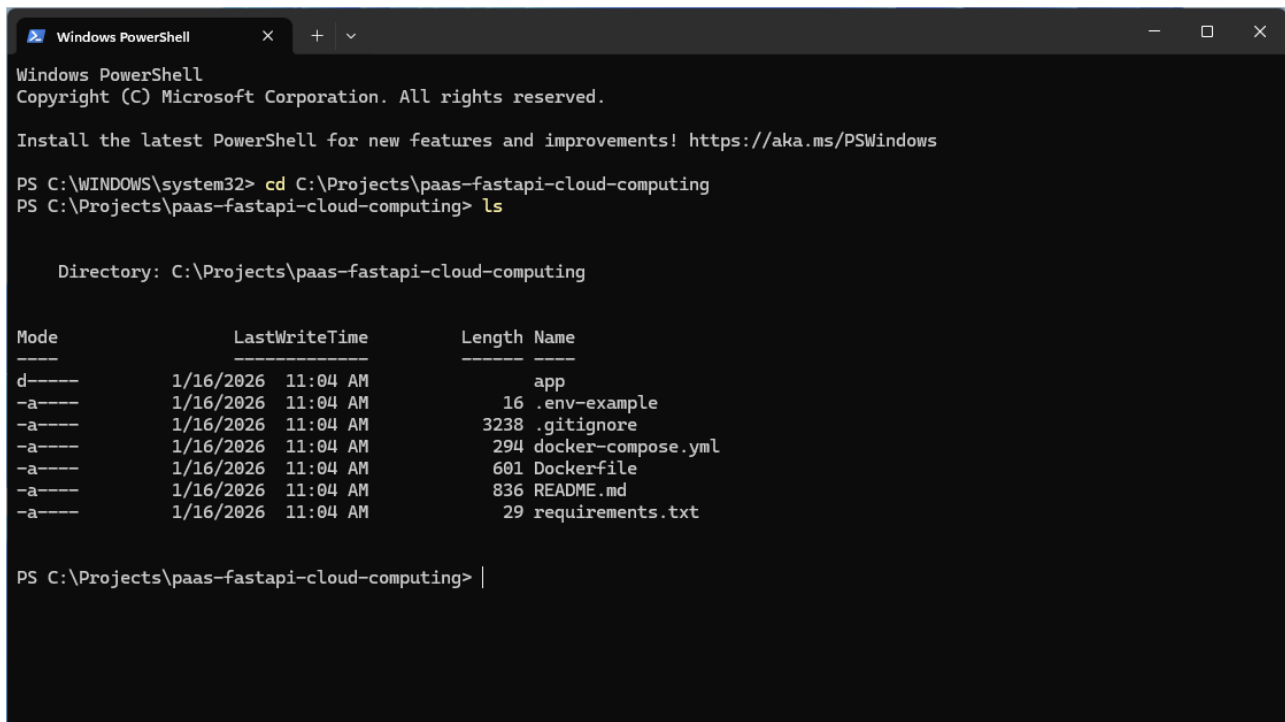


Figure 36 Project directory structure verified locally before running the application with Docker.

Assignment 2: PaaS with GitHub deployment (Full version)

Course: Cloud Computing & Data Engineering

By: Hamed Ahmadinia

```
Windows PowerShell
-----
d----- 1/16/2026 11:04 AM app
-a----- 1/16/2026 11:04 AM 16 .env-example
-a----- 1/16/2026 11:04 AM 3238 .gitignore
-a----- 1/16/2026 11:04 AM 294 docker-compose.yml
-a----- 1/16/2026 11:04 AM 601 Dockerfile
-a----- 1/16/2026 11:04 AM 836 README.md
-a----- 1/16/2026 11:04 AM 29 requirements.txt

PS C:\Projects\paas-fastapi-cloud-computing> docker-compose up --build
time="2026-01-16T11:57:11+02:00" level=warning msg="project has been loaded without an explicit name from a symlink. Usi
ng name \"paas-fastapi-cloud-computing\""
time="2026-01-16T11:57:11+02:00" level=warning msg="C:\\Projects\\paas-fastapi-cloud-computing\\docker-compose.yml: the
attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
#1 [internal] load local bake definitions
#1 reading from stdin 599B done
#1 DONE 0.0s

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 640B done
#2 DONE 0.0s

#3 [internal] load metadata for docker.io/library/python:3.10
#3 DONE 0.3s

#4 [internal] load .dockerignore
#4 transferring context: 2B done
#4 DONE 0.0s
```

Figure 37 Running the FastAPI application locally using Docker Compose with the `docker-compose up --build` command.

```
Windows PowerShell
-----
d----- 1/16/2026 11:04 AM app
-a----- 1/16/2026 11:04 AM 16 .env-example
-a----- 1/16/2026 11:04 AM 3238 .gitignore
-a----- 1/16/2026 11:04 AM 294 docker-compose.yml
-a----- 1/16/2026 11:04 AM 601 Dockerfile
-a----- 1/16/2026 11:04 AM 836 README.md
-a----- 1/16/2026 11:04 AM 29 requirements.txt

PS C:\Projects\paas-fastapi-cloud-computing> docker-compose up --build
time="2026-01-16T11:57:11+02:00" level=warning msg="project has been loaded without an explicit name from a symlink. Usi
ng name \"paas-fastapi-cloud-computing\""
time="2026-01-16T11:57:11+02:00" level=warning msg="C:\\Projects\\paas-fastapi-cloud-computing\\docker-compose.yml: the
attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
#1 [internal] load local bake definitions
#1 reading from stdin 599B done
#1 DONE 0.0s

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 640B done
#2 DONE 0.0s

#3 [internal] load metadata for docker.io/library/python:3.10
#3 DONE 0.3s

#4 [internal] load .dockerignore
#4 transferring context: 2B done
#4 DONE 0.0s
```

Figure 38 Running the FastAPI application locally using Docker Compose (`docker-compose up --build`).

Assignment 2: PaaS with GitHub deployment (Full version)

Course: Cloud Computing & Data Engineering

By: Hamed Ahmadinia

```
Windows PowerShell
#11 exporting config sha256:cd6385a8f66761891fd4936705a829b67019d490898f0b2512af30157aeb669b done
#11 exporting attestation manifest sha256:9275f0ae646bd080f576dfed6ad8b4d386c08272532f0fe5b1ecce85855eb402 0.0s done
#11 exporting manifest list sha256:3a5409c0e307abe911ecb9cbfcf3e36ab0f68349b6a6c4a1639d9314cde6e039 done
#11 naming to docker.io/library/paas-fastapi-cloud-computing-fastapi-app:latest done
#11 unpacking to docker.io/library/paas-fastapi-cloud-computing-fastapi-app:latest
#11 unpacking to docker.io/library/paas-fastapi-cloud-computing-fastapi-app:latest 0.5s done
#11 DONE 2.5s

#12 resolving provenance for metadata file
#12 DONE 0.0s
[+] up 3/3
  Image paas-fastapi-cloud-computing-fastapi-app Built 344.6s
  Network paas-fastapi-cloud-computing_default Created 0.0s
  Container paas-fastapi-cloud-computing-fastapi-app-1 Created 0.1s
Attaching to fastapi-app-1
fastapi-app-1 | FastAPI Starting production server 🚀
fastapi-app-1 | Searching for package file structure from directories with
fastapi-app-1 | __init__.py files
fastapi-app-1 | Importing from /code
fastapi-app-1 | module
fastapi-app-1 |   app
fastapi-app-1 |   |__init__.py
fastapi-app-1 |   |main.py
fastapi-app-1 | code
fastapi-app-1 |   Importing the FastAPI app object from the module with the following
fastapi-app-1 |   code:
fastapi-app-1 |   from app.main import app
fastapi-app-1 | app
fastapi-app-1 |   Using import string: app.main:app
fastapi-app-1 | server
fastapi-app-1 |   Server started at http://0.0.0.0:8080
fastapi-app-1 | server
fastapi-app-1 |   Documentation at http://0.0.0.0:8080/docs
fastapi-app-1 | Logs:
fastapi-app-1 | INFO Started server process [7]
fastapi-app-1 | INFO Waiting for application startup.
fastapi-app-1 | INFO Application startup complete.
fastapi-app-1 | INFO Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)

View in Docker Desktop View Config Enable Watch Detach
```

Figure 39 FastAPI application successfully running locally inside a Docker container using Docker Compose.

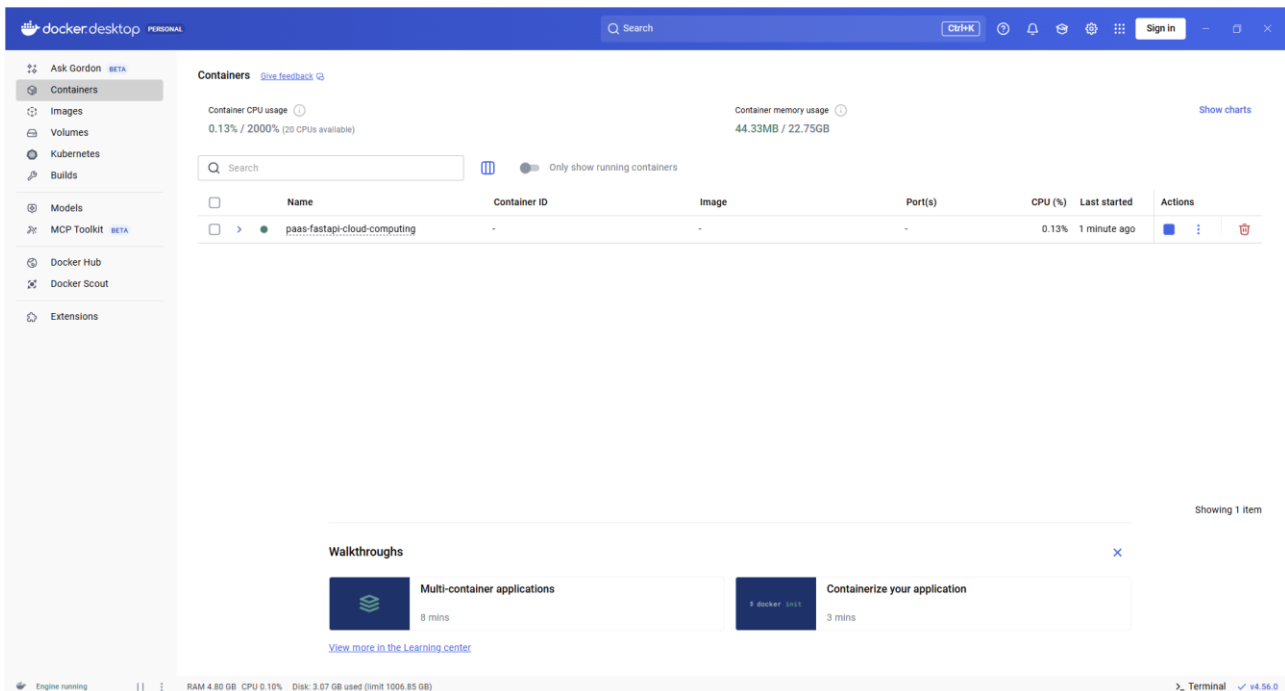


Figure 40 Docker Desktop showing the FastAPI container running locally after successful Docker Compose deployment.

Assignment 2: PaaS with GitHub deployment (Full version)

Course: Cloud Computing & Data Engineering

By: Hamed Ahmadinia

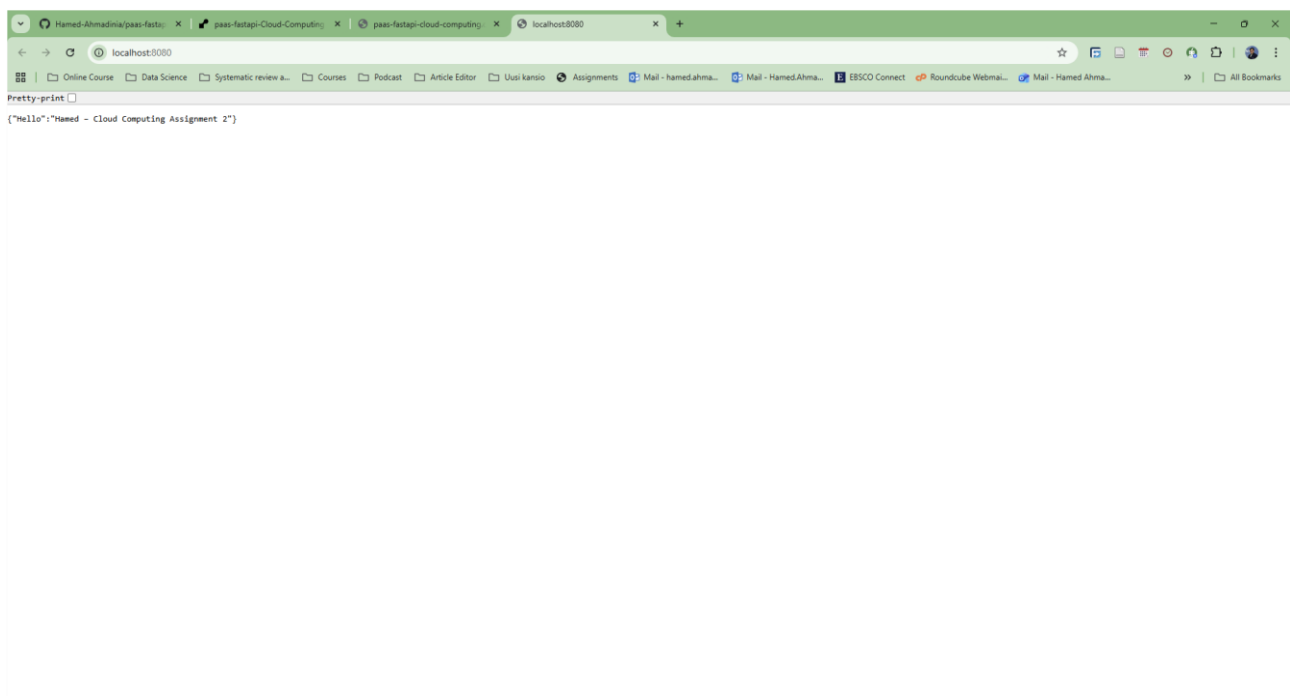


Figure 41 FastAPI application running locally via Docker; showing the modified Hello response at localhost:8080

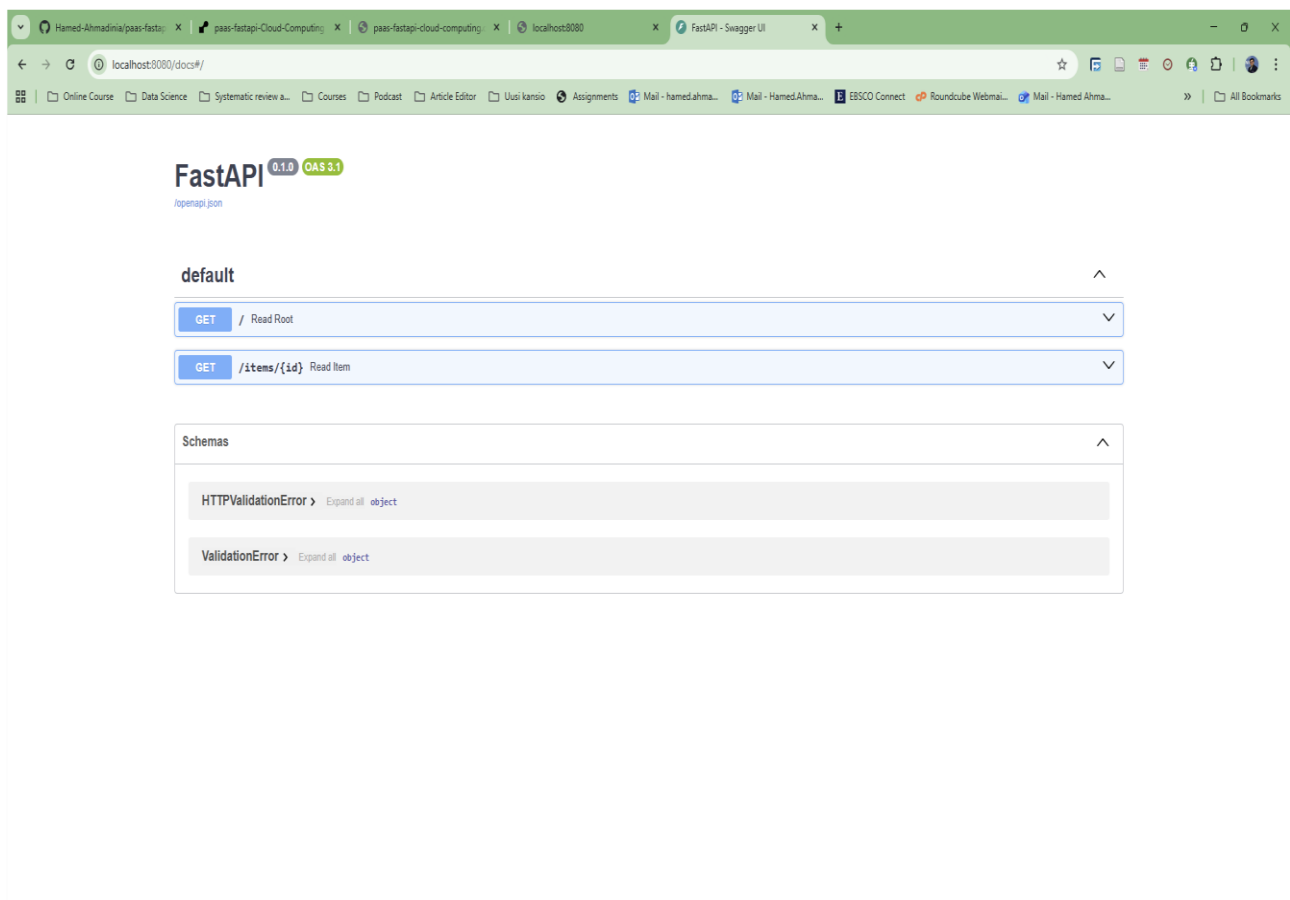


Figure 42 FastAPI Swagger UI (/docs) accessed locally while the application is running in Docker.

Assignment 2: PaaS with GitHub deployment (Full version)

Course: Cloud Computing & Data Engineering

By: Hamed Ahmadinia

Part F – Version Control and Continuous Deployment via GitHub: In this step, the project was managed using GitHub for version control. The FastAPI repository created from the provided template was connected to my local development environment using Git and SSH. All application code, Docker configuration files, and supporting files were tracked using Git.

After modifying the FastAPI application locally (including changing the Hello message and adding a compute endpoint), the changes were committed and pushed to the GitHub repository using standard Git commands. This push automatically triggered a new deployment on Render, demonstrating continuous deployment through GitHub integration.

Issue encountered: Initially, Render could not detect the repository when setting up the deployment because the required GitHub App permissions were not fully configured.

Resolution: This was resolved by updating the Render GitHub App permissions to allow access only to the selected repository. After granting access, the repository became visible in Render, and automatic deployments worked correctly. Each push to the repository resulted in a new successful deployment, and the updated API response was visible at the live Render URL.

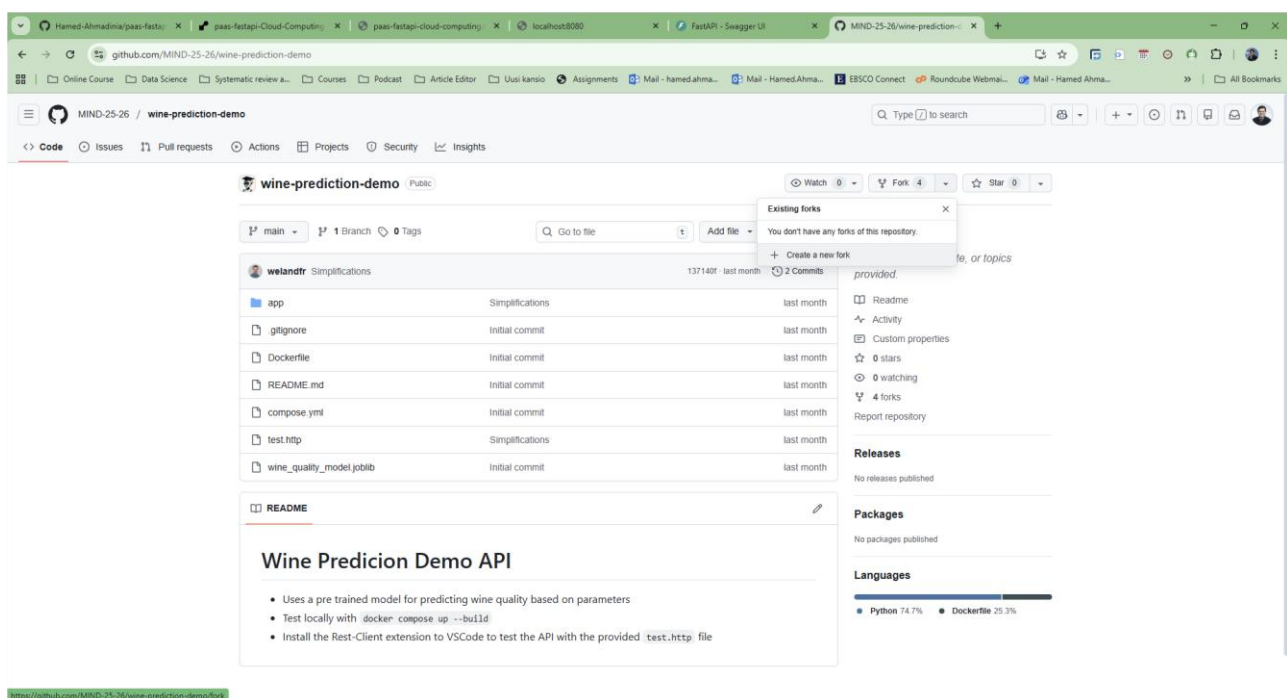


Figure 43 Original Wine Prediction Demo Repository on GitHub

Assignment 2: PaaS with GitHub deployment (Full version)

Course: Cloud Computing & Data Engineering

By: Hamed Ahmadinia

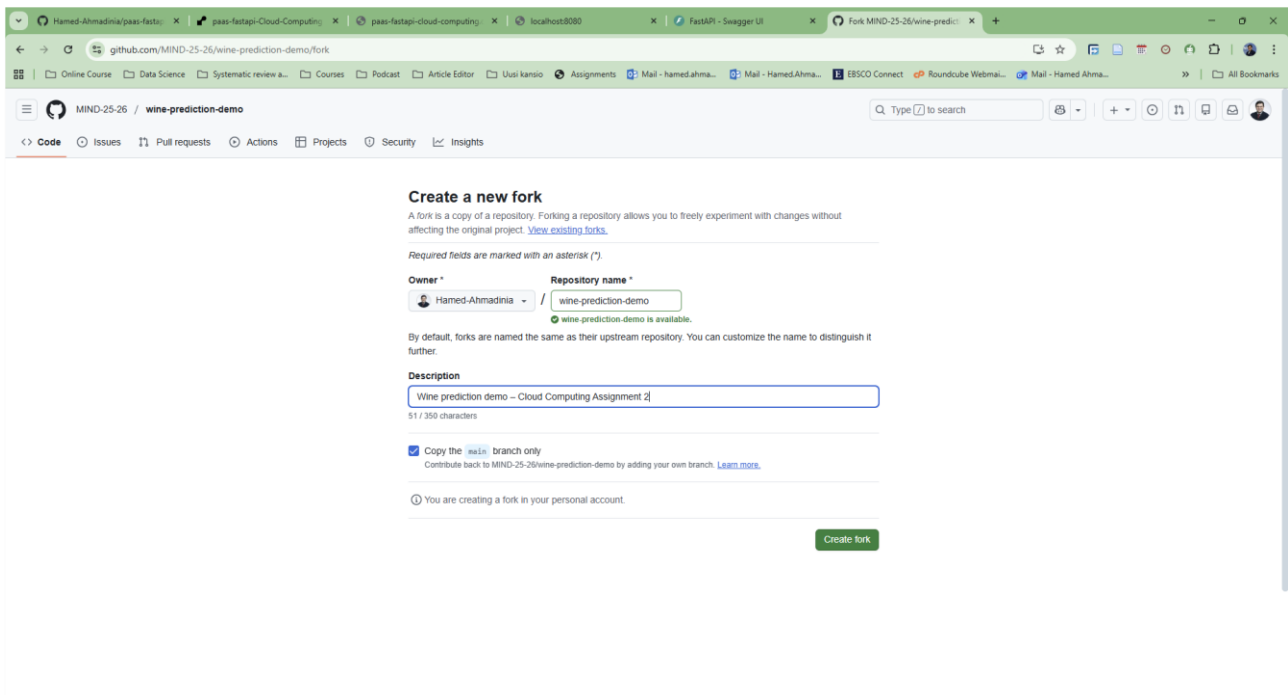


Figure 44 Forking the Wine Prediction Demo Repository on GitHub

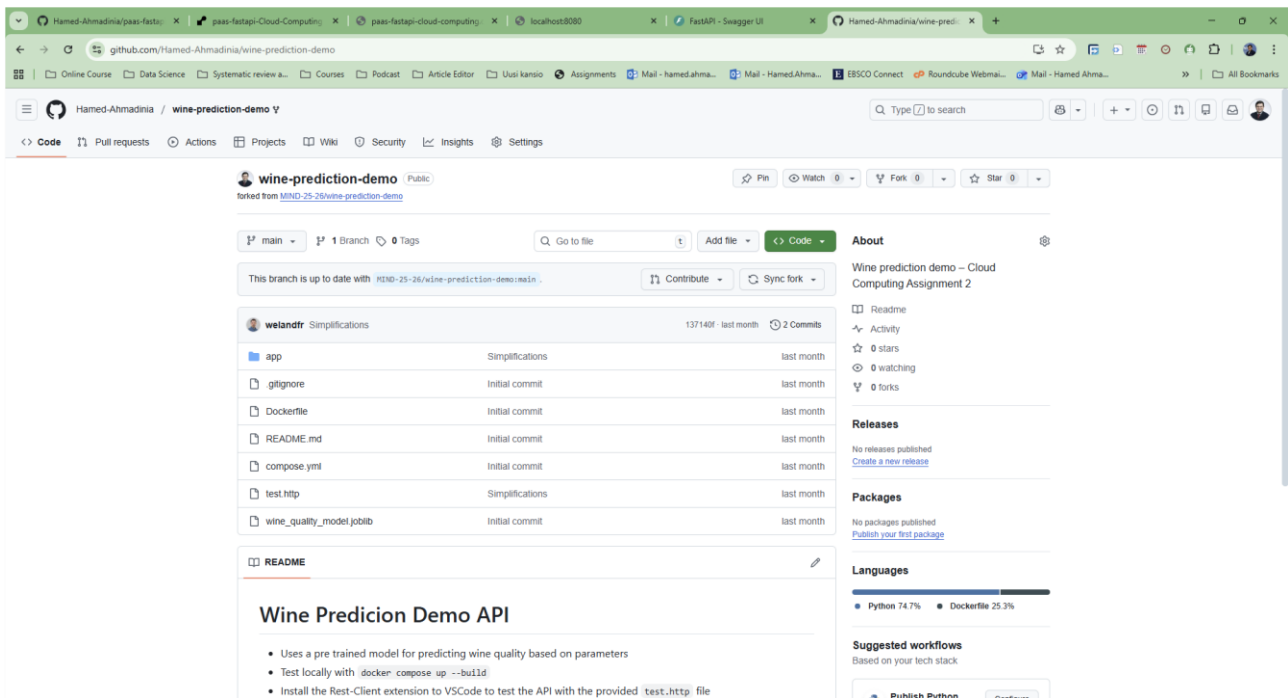


Figure 45 Forked Wine Prediction Repository Under Personal GitHub Account

Assignment 2: PaaS with GitHub deployment (Full version)

Course: Cloud Computing & Data Engineering

By: Hamed Ahmadiania

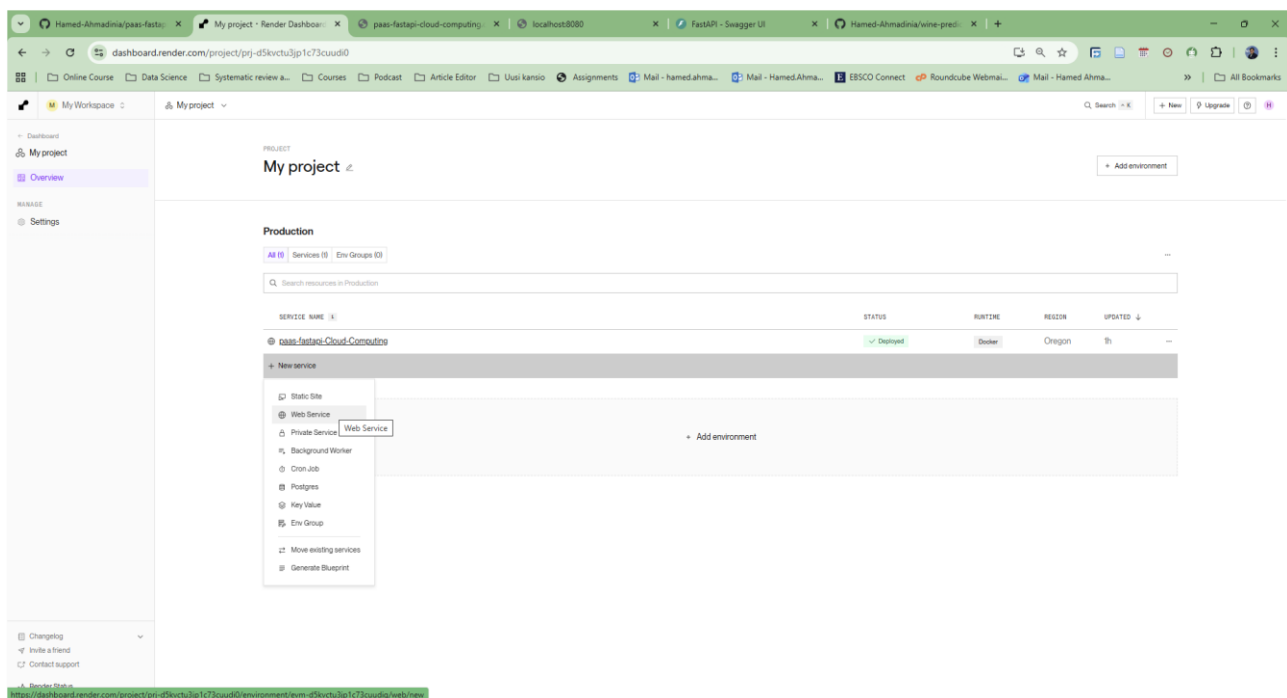


Figure 46 Creating a Docker-Based Web Service on Render

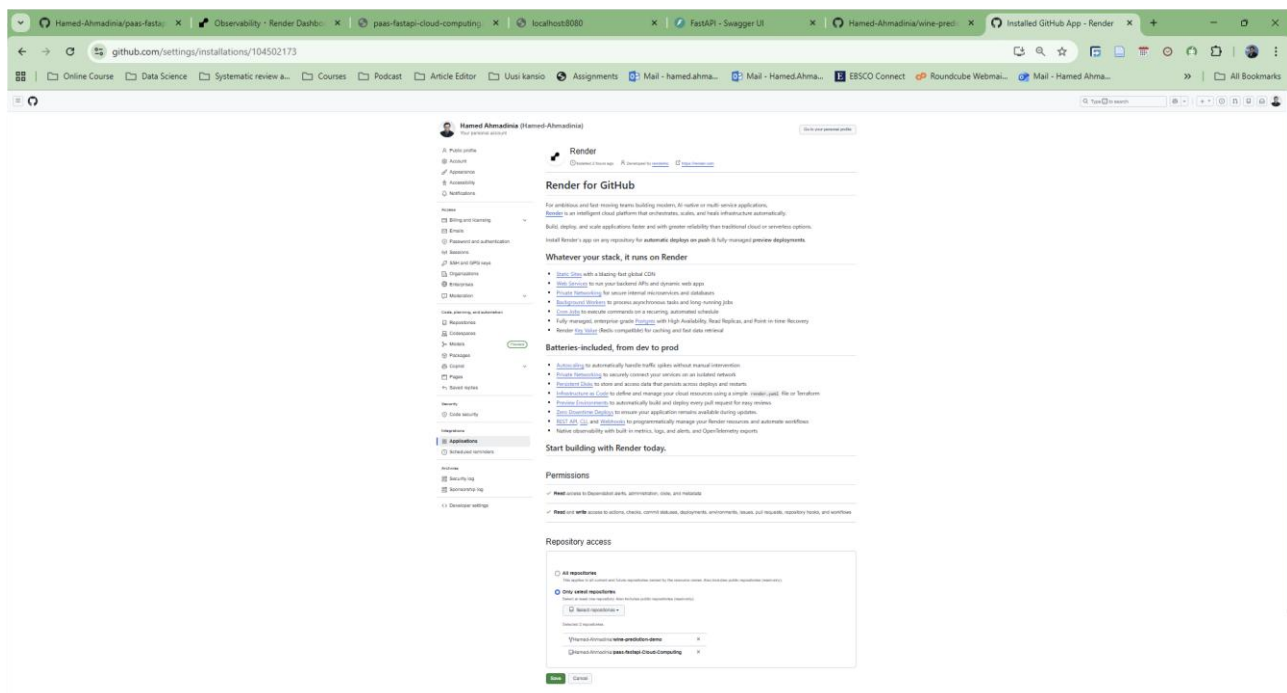


Figure 47 Configuring Render GitHub App Repository Permissions

Optional Part G – API Request Handling and Computation: In Optional Part G, the FastAPI application was extended to accept structured client requests and return computed responses. A new POST endpoint (`/compute`) was added to the application, allowing users to send numerical input values and receive calculated results. A Pydantic model (`ComputeRequest`) was used to define and validate the request body, ensuring that the input

Assignment 2: PaaS with GitHub deployment (Full version)

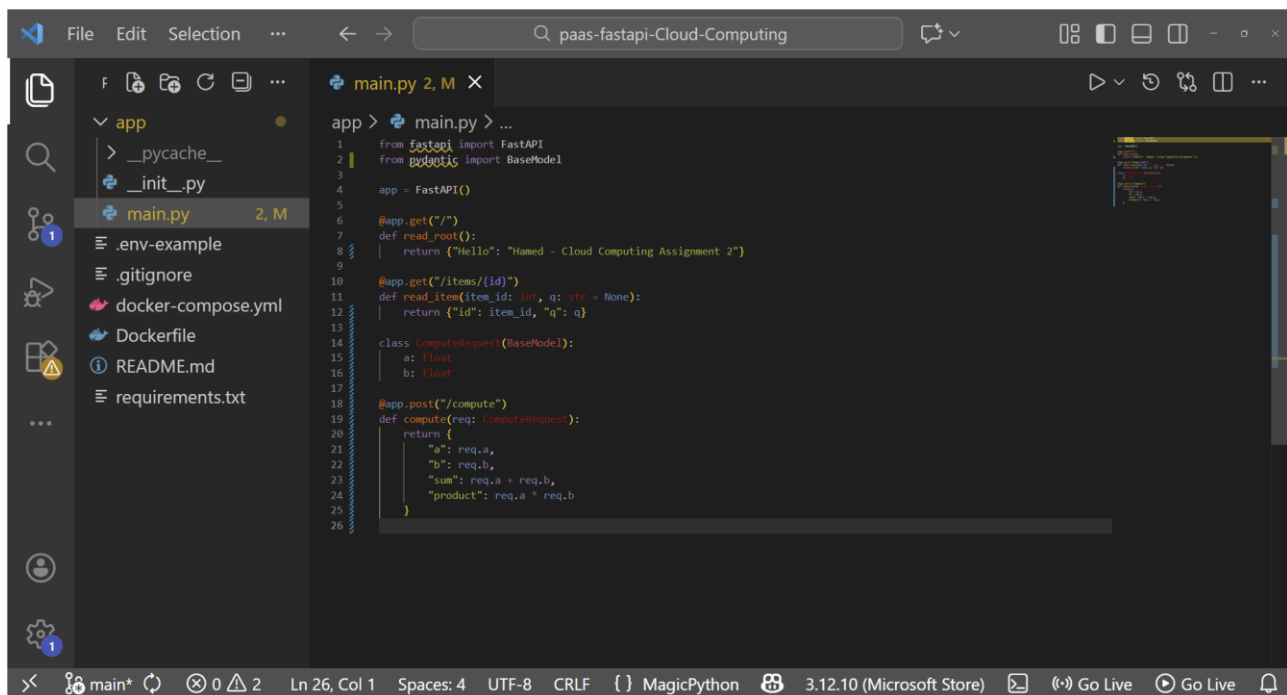
Course: Cloud Computing & Data Engineering

By: Hamed Ahmadiania

data was correctly structured before processing. The `/compute` endpoint was implemented to accept two numerical values (a and b) and return a JSON response containing the original inputs along with their sum and product. This demonstrated how FastAPI can be used not only to return static responses but also to perform server-side computations based on client-provided data. The new endpoint was tested using the FastAPI interactive Swagger UI (`/docs`). Test requests were sent by providing valid JSON input, and the API returned correct computed results in the response. Successful execution confirmed that request validation, computation logic, and response formatting were all working as expected, both locally and within the containerized environment.

Issue encountered: During initial testing, incorrect or incomplete JSON input caused validation errors when calling the `/compute` endpoint.

Resolution: This was resolved by carefully following the Pydantic schema and ensuring that all required fields were included with the correct data types in the request body. After correcting the input format, the endpoint responded successfully with the expected computed values.



```
1 from fastapi import FastAPI
2 from pydantic import BaseModel
3
4 app = FastAPI()
5
6 @app.get("/")
7 def read_root():
8     return {"Hello": "Hamed - Cloud Computing Assignment 2"}
9
10 @app.get("/items/{id}")
11 def read_item(item_id: int, q: str = None):
12     return {"id": item_id, "q": q}
13
14 class ComputeRequest(BaseModel):
15     a: float
16     b: float
17
18 @app.post("/compute")
19 def compute(req: ComputeRequest):
20     return {
21         "a": req.a,
22         "b": req.b,
23         "sum": req.a + req.b,
24         "product": req.a * req.b
25     }
```

Figure 48 Implementation of the `/compute` Endpoint in FastAPI

Assignment 2: PaaS with GitHub deployment (Full version)

Course: Cloud Computing & Data Engineering

By: Hamed Ahmadinia

```
Windows PowerShell
fastapi-app-1 | Logs:
fastapi-app-1 |
fastapi-app-1 | INFO Started server process [7]
fastapi-app-1 | INFO Waiting for application startup.
fastapi-app-1 | INFO Application startup complete.
fastapi-app-1 | INFO Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
fastapi-app-1 | INFO 172.18.0.1:45246 - "GET / HTTP/1.1" 200
fastapi-app-1 | INFO 172.18.0.1:45246 - "GET /favicon.ico HTTP/1.1" 404
fastapi-app-1 | INFO 172.18.0.1:45260 - "GET /docs HTTP/1.1" 200
fastapi-app-1 | INFO 172.18.0.1:45260 - "GET /openapi.json HTTP/1.1" 200
fastapi-app-1 | INFO 172.18.0.1:60422 - "GET / HTTP/1.1" 200
Gracefully Stopping... press Ctrl+C again to force
Container paas-fastapi-cloud-computing-fastapi-app-1 Stopping
Container paas-fastapi-cloud-computing-fastapi-app-1 Stopped

PS C:\Projects\paas-fastapi-cloud-computing>
```

Figure 49 Local Container Execution Logs for the FastAPI Application

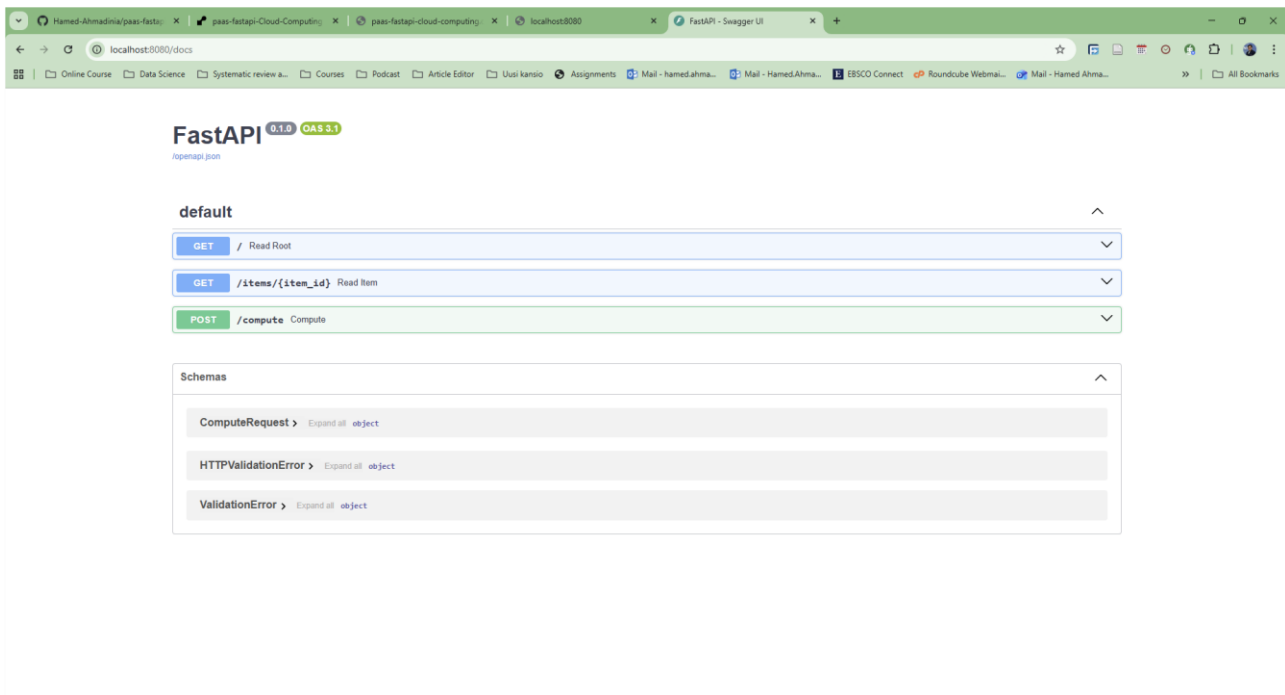


Figure 50 FastAPI Swagger UI Showing Available API Endpoints

Assignment 2: PaaS with GitHub deployment (Full version)

Course: Cloud Computing & Data Engineering

By: Hamed Ahmadiania

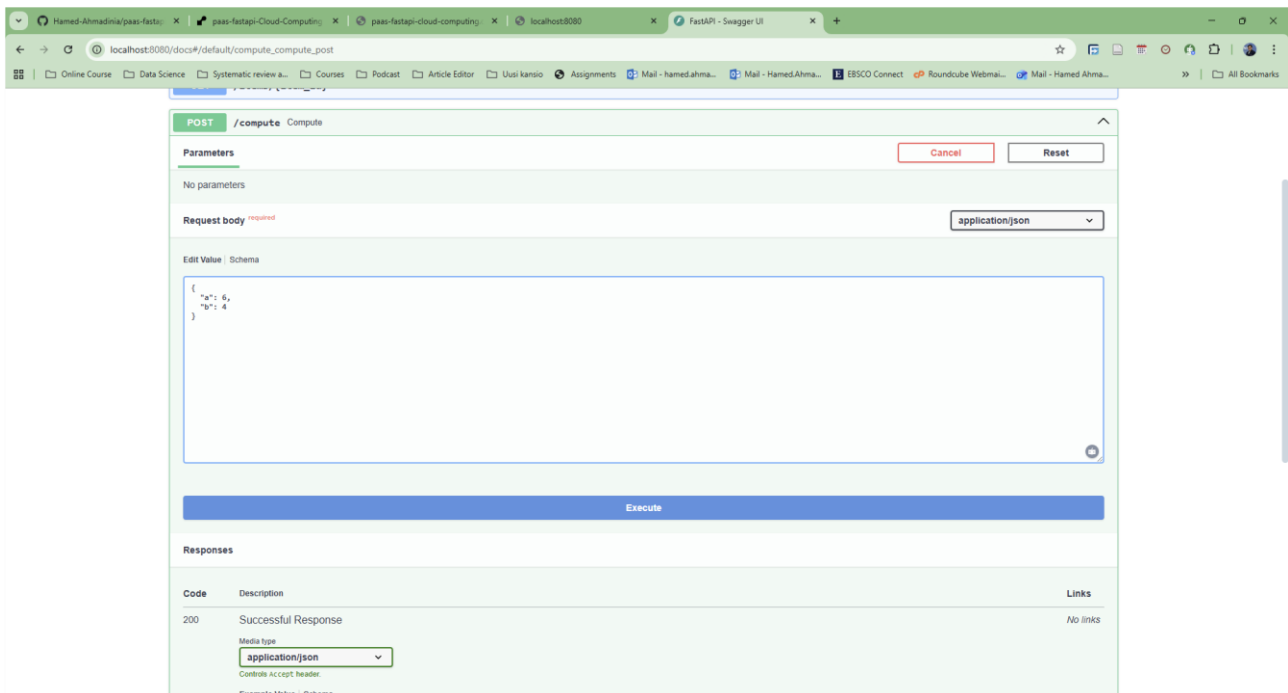


Figure 51 Submitting a POST Request to the /compute Endpoint via Swagger UI

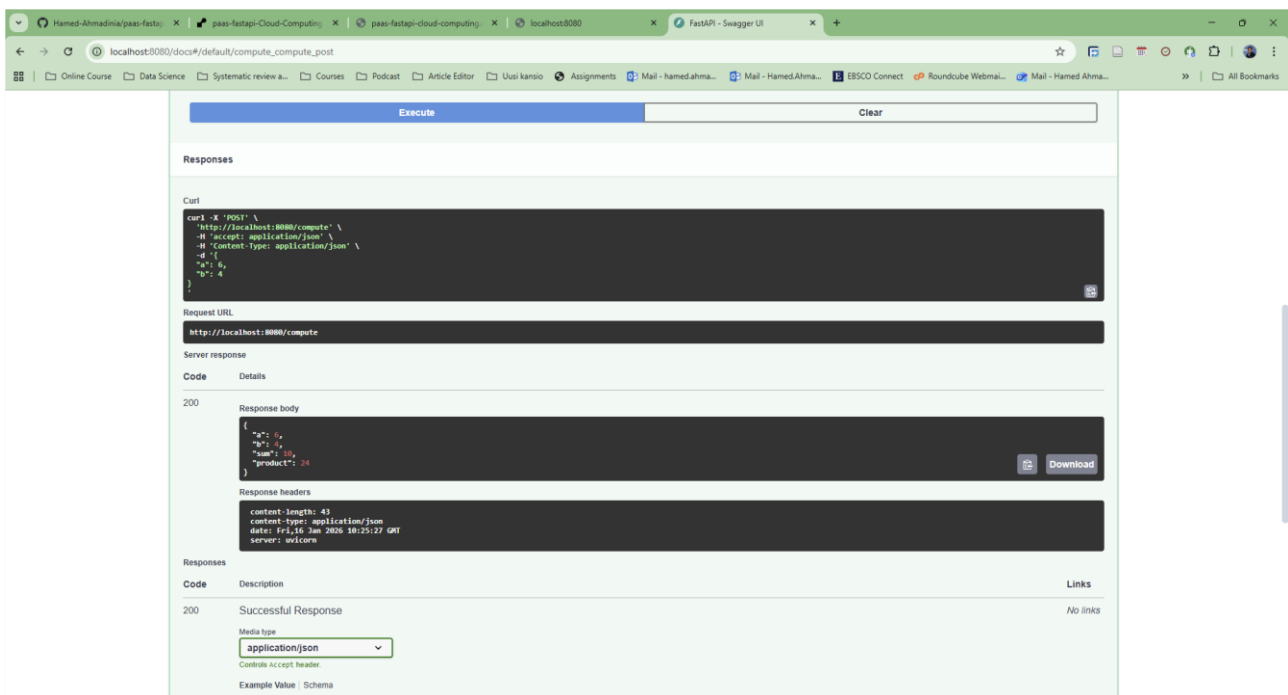


Figure 52 Successful Computation Response from the /compute Endpoint

Optional Part H – Wine Prediction Demo (Machine Learning Deployment on Render): In Optional Part H, a machine learning-based FastAPI application was deployed to the cloud using Render as a Platform-as-a-Service (PaaS) provider. This part of the assignment was implemented using the provided wine-prediction-demo repository as a basis. The application integrates a pre-trained machine learning model and exposes a prediction endpoint that

Assignment 2: PaaS with GitHub deployment (Full version)

Course: Cloud Computing & Data Engineering

By: Hamed Ahmadinia

accepts numerical input features and returns predicted wine quality values. A separate GitHub repository was created for the wine prediction service, and the project was deployed to Render using Docker in the same manner as the core FastAPI application. The repository was connected to Render via GitHub, and a Docker-based web service was configured. After deployment, Render automatically built the Docker image and launched the service. The prediction API was tested both locally and in the cloud. The FastAPI Swagger UI (/docs) was used to submit test requests to the prediction endpoint, and valid prediction results were returned. In addition, HTTP requests were tested using the provided tools, confirming correct request handling and response generation.

Issue encountered: During deployment, a version mismatch warning occurred when loading the pre-trained machine learning model. This warning was caused by differences between the scikit-learn version used during model training and the version installed in the deployment environment.

Resolution: The issue was addressed by installing a compatible scikit-learn version in the Docker environment. Despite the warning, the model loaded correctly, and prediction results were validated through testing. The application successfully returned correct inference outputs, confirming that the machine learning integration and deployment were functioning as intended. Render deployment logs confirmed that the service was live and operational. The deployed application exposes a public API endpoint for predictions, provides interactive documentation via the Swagger UI, and demonstrates the successful integration of a machine learning model into a cloud-deployed FastAPI service.

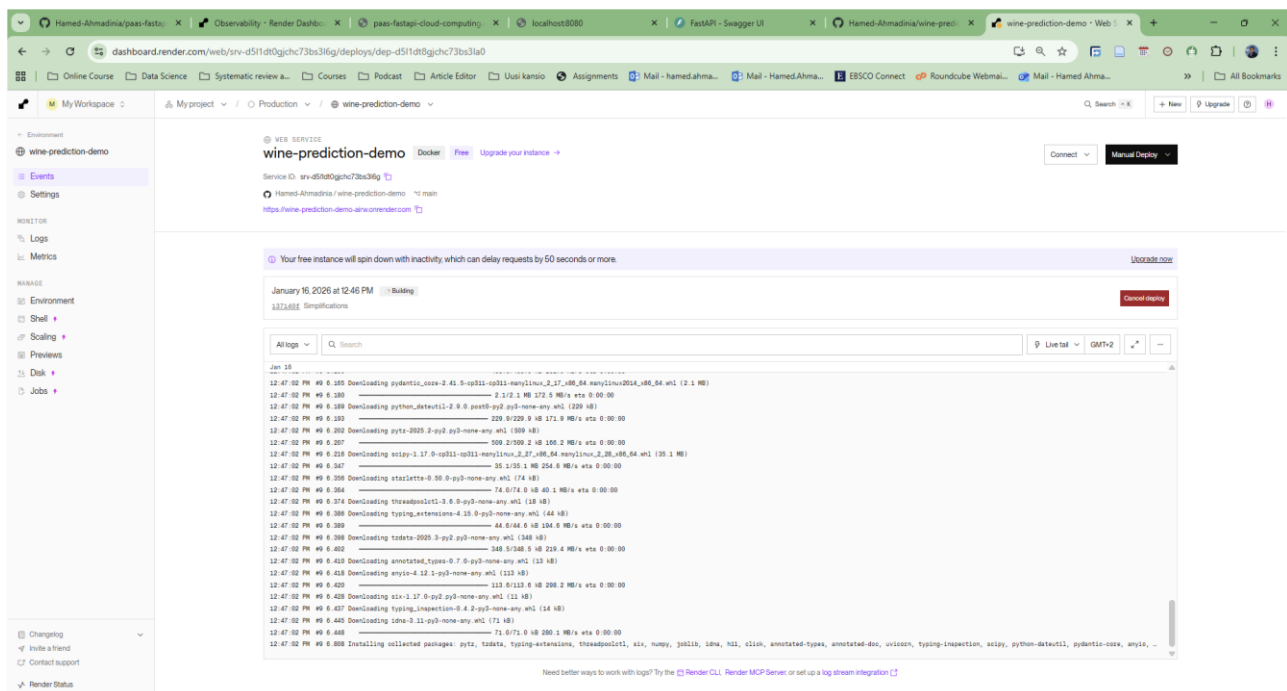


Figure 53 Deployment Logs for the Wine Prediction Demo on Render

Assignment 2: PaaS with GitHub deployment (Full version)

Course: Cloud Computing & Data Engineering

By: Hamed Ahmadinia

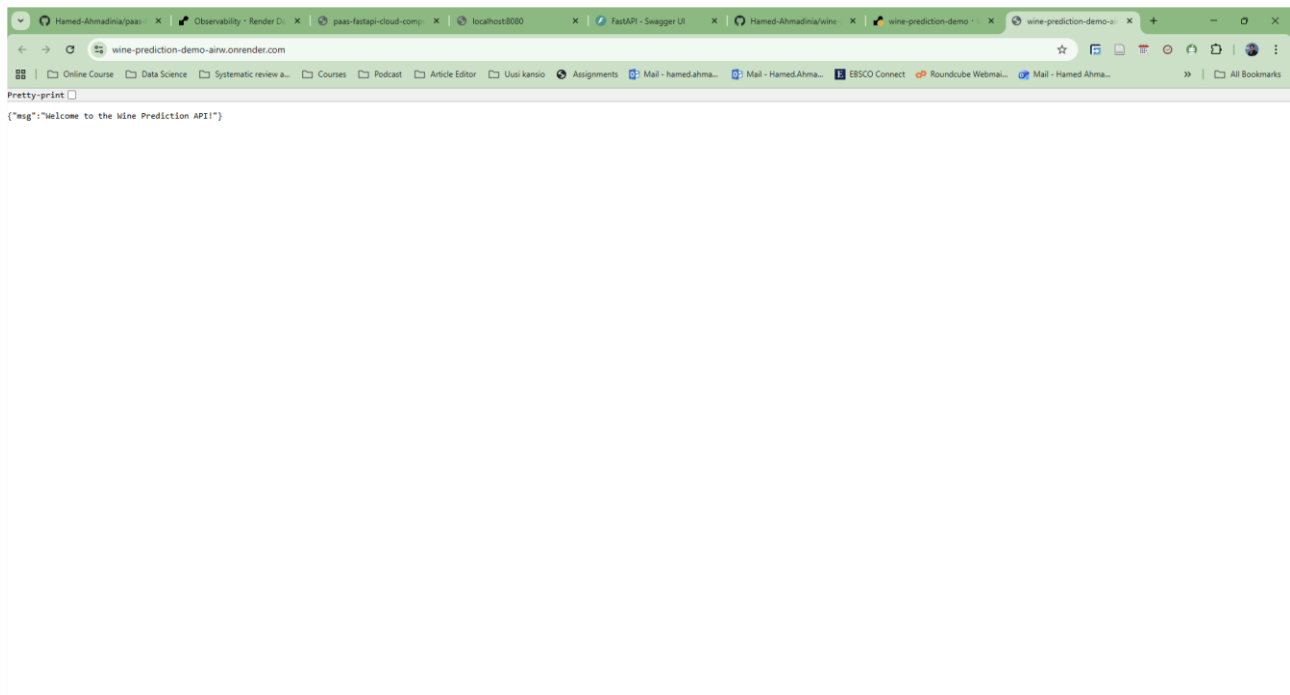


Figure 54 Live Wine Prediction API Accessible via Public Render URL

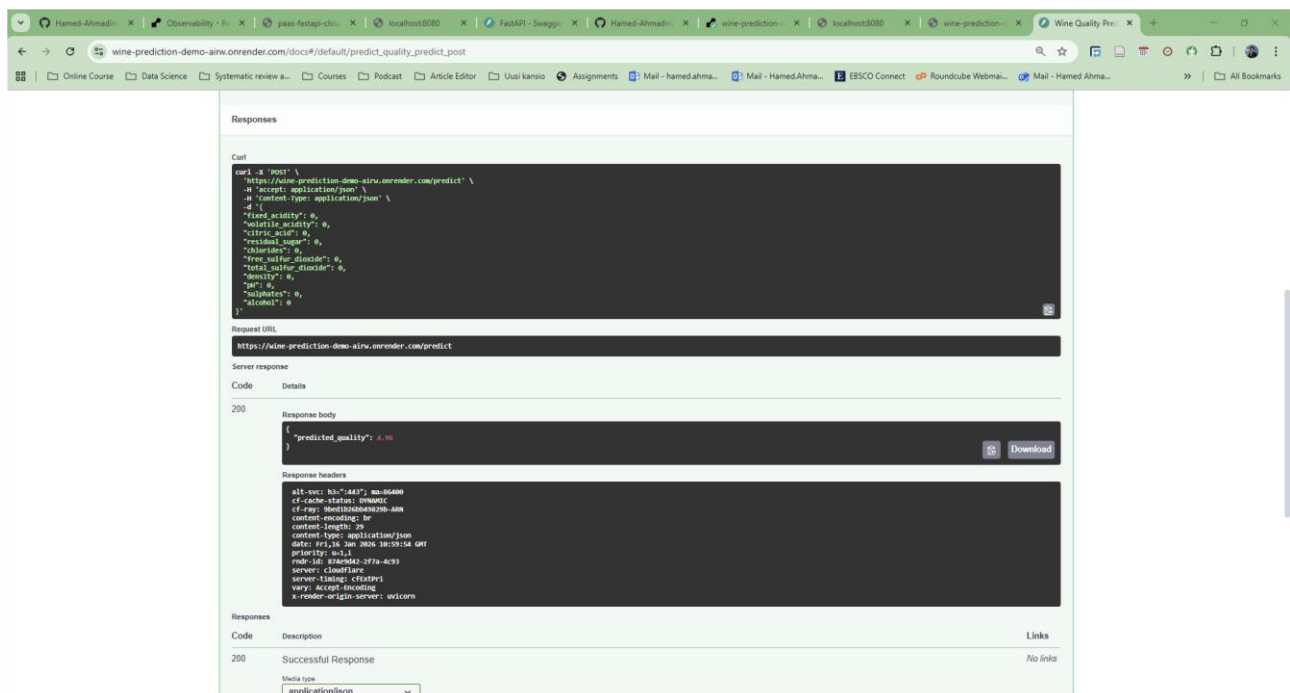


Figure 55 Successful Wine Quality Prediction Using the Deployed ML API

Final Conclusion

This assignment demonstrates the complete lifecycle of a cloud-ready machine learning application, from FastAPI development and local containerization to continuous deployment via GitHub and public hosting on a PaaS platform. Throughout the implementation, multiple technical challenges related to containerization, permissions, and dependency management

Assignment 2: PaaS with GitHub deployment (Full version)**Course:** Cloud Computing & Data Engineering**By:** Hamed Ahmadinia

were encountered and resolved through systematic debugging and configuration adjustments. The final application is fully functional, reproducible, and publicly accessible, satisfying all technical, deployment, and automation requirements of Assignment 2.